

Contents

SASA and the Cost of Self-Discipline	3
0. Master table of dated work	3
2. The artefact under review	13
2.1 What exists	13
2.2 The work record	13
3. Formal statement of the implemented decoder	15
4. Claim-to-evidence audit	15
5. Analytical results	16
5.1 Theorem 1 — the decoder is context-independent	16
5.2 Theorem 2 — the margin term is rank-one, and the arithmetic collapses	17
5.3 Theorem 3 — affine estimators cannot restore context-dependence	18
6. Original research agenda	18
6.1 NLMA: a non-linear next-state margin approximation	20
6.2 The closed-loop evaluation trap	20
6.3 Geometry: interference, depth, and rank	21
6.4 Transfer	21
6.5 Systems	21
6.6 Schedules are dose control, not policy	21
7. Experimental protocol	22
8. Results	22
9. Discussion and threats to validity	27
10. Conclusion	27
Appendices	28
Appendix B — The margin is rank-one: exactness, cost, and context-independence	28
B.1 Claim	28
B.2 What is measured, and what is expected	28
B.3 Results	29
B.4 Cost	30
B.5 What this does and does not show	30
Appendix C — Fixing the estimator: NLMA, and what “a good subspace” requires	31
C.1 Why an affine estimator cannot help	31
C.2 NLMA	31
C.3 The regime that actually bites: $N \ll d$	31
C.4 α is not scale-free, and that is the practical bug	32
C.5 Threats	33
Appendix D — A pinned external judge, and the closed-loop evaluation trap	33
D.1 The structural problem	33
D.2 The pinned judge	33
D.3 Protocol, fixed before the run	34
D.4 Results	34
D.5 The headline negative result	35
D.6 Threats	35

Appendix E — Geometry: depth, rank, and the price of static interference	35
E.1 Layer sweep: the toxicity subspace is not identifiable	35
E.2 Two directions, two conclusions, and why the second is the weaker one	36
E.3 Static interference prediction (idea 6) — specified, not measured	37
E.4 Rank- k subspaces (idea 8) — specified, not measured	37
E.5 What would change these conclusions	37
Appendix F — Cross-model transfer, and a feasibility correction to Phase 3	37
F.1 Scope, stated before the result	37
F.2 Setup	38
F.3 Results	38
F.4 Threats	38
Appendix G — Reproducibility, provenance, and how to re-run this	39
G.1 Environment	39
G.2 Commands	39
G.3 Determinism and caching	39
G.4 What is cached, and what that implies	40
G.5 Threats to reproducibility that remain	40
G.6 Repository hygiene	40
Appendix H — Graded attributes, interference geometry, and confounds	41
H.1 Why the attribute set is not toxicity	41
H.2 Confound geometry	41
H.3 Static interference prediction: right sign, no power	41
H.4 Graded supervision: free, and measurably better	42
H.5 Threats	43
Appendix I — Steering schedules, the U2 rule, and the degeneracy knee	43
I.1 What a schedule can and cannot change here	43
I.2 Schedules: no measurable difference at the literature’s strengths	43
I.3 The degeneracy knee	43
I.4 The U2 rule was exercised, and its verdict is not evidence	44
I.5 Threats	44
Appendix J — Where subspace transfer breaks	45
J.1 Setup	45
J.2 Results	45
J.3 Reading	46
J.4 Threats	46
J.5 What would settle it	46
Appendix K — The decode loop: caching and batching	47
K.1 Why this is the real cost	47
K.2 Caching: exact, and the analytic gain is not yet realised	47
K.3 Batching: correctness yes, throughput only once vectorised	47
K.4 Threats	48
Appendix L — Is the pinned judge adversarially fooled?	48
L.1 Design	48
L.2 The attack was applied	48
L.3 Results	49
L.4 The honest reading	49
L.5 What Appendix D’s null result now means	49
L.6 How to make this testable	50

SASA and the Cost of Self-Discipline

A dated audit of hwilner/llm-self-detoxifier, a formal analysis of margin-steered decoding, and seven original research contributions

Report generated: 2026-09-30 Repository: <https://github.com/hwilner/llm-self-detoxifier> @ main · baseline commit 9ab2a2c Work items in this report: 48 (15 commits, 29 task cards, 4 pull requests), spanning 2025-11-22 → 2026-09-30

0. Master table of dated work

Every row is one verifiable, dated artefact in the repository’s history: a commit, a task card, or a pull request opened while producing this report. The table is generated directly from the GitHub API and the git log, so it can be regenerated and diffed at any time. Rows 1–48 are ordered chronologically; the pull requests near the end are this report’s own contributions, each of which has a matching appendix section with its full write-up.

Provenance note. At the baseline commit the repository contained **zero pull requests**. All upstream work is tracked as issue *task cards* (hwilner created all 29 of them in a single burst on 2026-09-22). “PR date” in the request is therefore reported as *work-item date*, with the artefact type stated explicitly in every row. Pull requests begin appearing in this report’s own contributions.

#	Date	Work item	Phase · Size	Evidence	Status
1	2025-11-22	commit bd65b24 Initial implementation of SASA for LLM detoxification	Implementation	+1816/-0 in 13 files	merged
2	2026-09-22	commit 134d2b0 chore: add issue task-card template	Docs	+28/-0 in 1 file	merged
3	2026-09-22	commit 2097e92 docs: add METHODS.md — done/intended/undecided methods record	Docs	+84/-0 in 1 file	merged
4	2026-09-22	commit 2f84ac2 docs: add INTRODUCTION.md — ground-up SASA explainer	Docs	+99/-0 in 1 file	merged
5	2026-09-22	commit 3b3419c docs: add concept figure (SVG)	Implementation	+1/-0 in 1 file	merged
6	2026-09-22	commit 666e662 docs: add EX-TENDED_INTRODUCTION.md — zero-background explainer	Docs	+142/-0 in 1 file	merged
7	2026-09-22	commit 7169fe2 docs: add CONTRIBUTING.md newcomer guide	Docs	+47/-0 in 1 file	merged
8	2026-09-22	commit 7a192f2 docs: rewrite EX-TENDED_INTRODUCTION with zero-prerequisite numeric examples + figure embed	Docs	+150/-44 in 1 file	merged
9	2026-09-22	commit 9ab2a2c Clarify public project boundaries	Docs	+4/-4 in 2 files	merged

#	Date	Work item	Phase · Size	Evidence	Status
10	2026-09-22	commit 9db2735 docs: add PROJECT.md in-repo backlog board	Docs	+70/-0 in 1 file	merged
11	2026-09-22	commit addba8d docs: add Mermaid fallback for concept figure	Docs	+24/-0 in 1 file	merged
12	2026-09-22	commit bdeddd8 chore: add pull request template	Docs	+26/-0 in 1 file	merged
13	2026-09-22	commit c78acea docs: add ROADMAP.md — phased plan incl. TSM-MA	Docs	+104/-0 in 1 file	merged
14	2026-09-22	commit dc5e9c9 Document refined contributor child tasks	Docs	+24/-0 in 1 file	merged
15	2026-09-22	commit fec98b4 docs: embed concept figure in INTRODUCTION	Docs	+4/-0 in 1 file	merged
16	2026-09-22	card #1 [L] Phase 1: Evaluation hardening	Phase 1 · L	evaluation, 6 acceptance criteria	open
17	2026-09-22	card #2 [L] Phase 2: Multi-model support and efficiency	Phase 2 · L	code, 6 acceptance criteria	open
18	2026-09-22	card #3 [L] Phase 3 (novel research): TSM-MA — transferred subspace margins with multi-attribute scheduling	Phase 3 (TSM-MA) · L	experiment, 10 acceptance criteria	open
19	2026-09-22	card #4 [S] Add BOLD bias + sycophancy probe evaluation harness	Phase 1 · S	evaluation, 4 acceptance criteria	open
20	2026-09-22	card #5 [S] Human spot-check protocol doc + inter-annotator sheet	Phase 1 · S	docs, 4 acceptance criteria	open
21	2026-09-22	card #6 [XS] Full benchmark re-run with pre-registered metrics + honest report	Phase 1 · XS	experiment, 4 acceptance criteria	open
22	2026-09-22	card #7 [S] Alpha scheduling module (fixed/linear/cosine) + tests	Phase 2 · S	code, 4 acceptance criteria	open
23	2026-09-22	card #8 [S] Llama-family hidden-state adapter + tests	Phase 2 · S	code, 4 acceptance criteria	open
24	2026-09-22	card #9 [XS] Latency/overhead benchmark table vs BaselineSampler	Phase 2 · XS	evaluation, 3 acceptance criteria	open
25	2026-09-22	card #10 [S] Expose subspace as serializable module + reproduce SASA on GPT-2-L and Llama-3.1-8B	Phase 3 (TSM-MA) · S	code, 4 acceptance criteria	open

#	Date	Work item	Phase · Size	Evidence	Status
26	2026-09-22	card #11 [S] Procrustes alignment map between paired hidden states of proxy/target + synthetic tests	Phase 3 (TSM-MA) · S	code, 4 acceptance criteria	open
27	2026-09-22	card #12 [S] Small→large transfer experiment: Llama-3.2-1B subspace → 8B decoding, vs direct-learned baseline	Phase 3 (TSM-MA) · S	experiment, 4 acceptance criteria	open
28	2026-09-22	card #13 [S] Multi-attribute margin composition (toxicity + sycophancy + bias) with per-attribute alpha schedules + interference study	Phase 3 (TSM-MA) · S	experiment, 4 acceptance criteria	open
29	2026-09-22	card #14 [S] Results report: transfer gap $\leq 15\%$ criterion, fluency cost, compute comparison, honest failure modes	Phase 3 (TSM-MA) · S	docs, 4 acceptance criteria	open
30	2026-09-22	card #15 [S] docs/API_REFERENCE.md for Subspace-Learner/SASASampler/BaselineSampler	Phase 0 · S	docs, 4 acceptance criteria	open
31	2026-09-22	card #16 [XS] Example notebook/script: detoxify GPT-2 on 5 prompts end-to-end	Phase 0 · XS	docs, 4 acceptance criteria	open
32	2026-09-22	card #17 [XS] pyproject.toml packaging cleanup + pytest config	Phase 0 · XS	code, 4 acceptance criteria	open
33	2026-09-22	card #18 [XS] Frozen benchmark configuration and deterministic rerun command	Phase 1 · XS	implementation, 3 acceptance criteria	open
34	2026-09-22	card #19 [XS] Benchmark execution and automatic-plus-human evaluation record collection	Phase 1 · XS	implementation, 3 acceptance criteria	open
35	2026-09-22	card #20 [XS] Honest benchmark results report with signed-rank and bootstrap outputs	Phase 1 · XS	implementation, 3 acceptance criteria	open
36	2026-09-22	card #21 [XS] Serializable subspace format with save-load round-trip tests	Phase 3 (TSM-MA) · XS	implementation, 3 acceptance criteria	open

#	Date	Work item	Phase · Size	Evidence	Status
37	2026-09-22	card #22 [XS] GPT-2-L SASA smoke reproduction with recorded seed and configuration	Phase 3 (TSM-MA) · XS	implementation, 3 acceptance criteria	open
38	2026-09-22	card #23 [XS] Llama-3.1-8B SASA smoke reproduction with recorded seed and configuration	Phase 3 (TSM-MA) · XS	implementation, 3 acceptance criteria	open
39	2026-09-22	card #24 [XS] Deterministic paired hidden-state collection with provenance	Phase 3 (TSM-MA) · XS	implementation, 3 acceptance criteria	open
40	2026-09-22	card #25 [XS] Transfer, direct, and identity-or-PCA baseline decoding runs	Phase 3 (TSM-MA) · XS	implementation, 3 acceptance criteria	open
41	2026-09-22	card #26 [XS] Locked statistical and perplexity result table generation	Phase 3 (TSM-MA) · XS	implementation, 3 acceptance criteria	open
42	2026-09-22	card #27 [XS] Multi-margin composition API with unit tests	Phase 3 (TSM-MA) · XS	implementation, 3 acceptance criteria	open
43	2026-09-22	card #28 [XS] Per-attribute schedule and attribute-margin fixtures	Phase 3 (TSM-MA) · XS	implementation, 3 acceptance criteria	open
44	2026-09-22	card #29 [S] Multi-attribute interference-study runner and outputs	Phase 3 (TSM-MA) · S	implementation, 3 acceptance criteria	open
45	2026-09-30	PR #30 feat(sasa): rank-one margin basis, exact static steering, and stable fitting	Original research (this report)	→ main, open	open
46	2026-09-30	PR #31 feat(evaluation, sasa): pinned external judge, NLMA next-state model, stable subspace fitting, four experiments	Original research (this report)	→ research/fast-margin-and-numerics (stacked), open	open
47	2026-09-30	PR #32 docs: add the dated research report, figures, and raw result data	Original research (this report)	→ research/evaluation-and-nlma (stacked), open	open
48	2026-09-30	PR #33 Implement remaining research ideas: schedules, multi-attribute, decoding, transfer boundary	Original research (this report)	+15072/-0, → main, open	open

Table 1. Dated work log for hwilner/llm-self-detoxifier, 2025-11-22 to 2026-09-30. Evidence column summarises

the verifiable footprint of each item (insertions/deletions, acceptance criteria, merge state). Generated by `build_report.py` from `data/work_items.json`.

1.
Intro-
duc-
tion

Detoxifying

a lan-
guage
model

at
infer-
ence
time,
with-
out

re-
train-
ing it
and

with-
out an
exter-
nal

re-
ward
model,

is an
attrac-
tive

engi-
neer-
ing
target.

SASA

(Self-
disciplined

Au-
tore-
gres-
sive

Sam-
pling;

Ko et

al.,

2024,

arXiv:2410.03818)

pro-
poses

such a
method:

learn a
linear

sub-
space

that
sepa-
rates

toxic
from
non-

toxic
con-
tent⁸ in

a
model's

hwilner/llm-self-detoxifier

is a public reim-ple-mentation of that idea. This report does three things. First, it audits the repository as a dated work record: what was built, when, and how much of the stated ambition is actually backed by code or measurement (Table 1, Figures 1–4, 7). Second, it analyses the implemented decoder and⁹ proves two

The
cen-
tral
claim
of this
report
is nar-
row
and
check-
able:

> Under the next-state estimator implemented in the reference code, SASA's > per-token margin is a constant vector plus a constant scalar. The scalar > cancels in the softmax, so the decoder reduces exactly to a static > vocabulary logit bias with no context dependence whatsoever — and the > per-

This is
not a
claim
that
SASA
fails.
A
static
toxicity-
weighted
logit
bias is
a per-
fectly
rea-
son-
able
decoding-
time
con-
troller,
and it
is
plausi-
bly
most
of
what
the
cur-
rent
imple-
menta-
tion
actu-
ally
deliv-
ers.
The
claim
is that
the
mech-
anism
the
paper
de-
scribes
and
the
mech-
anism
the
code
imple-
ments
have
di-¹²
verged,
that

2. The artefact under review

2.1 What exists

At the baseline commit 9ab2a2c the repository is 24 tracked files and 2 574 lines, of which 1 143 are Python. The algorithmic core is small:

Component	File	Lines	Role
SubspaceLearner	sasa/subspace_learner.py	281	Fits shared-covariance Gaussian, derives (w, b) in closed form
SASASampler	sasa/sampler.py	361	Per-token margin, logit adjustment, top-k/top-p, decode loop
BaselineSampler	sasa/sampler.py	142	Standard sampling, same interface, no margin term
tests/test_sasa.py	tests/test_sasa.py	299	15 unit tests, all on synthetic Gaussian clusters
Documentation	docs/*.md, README.md	1 274	Introduction, methods record, roadmap, architecture, testing
Backlog	29 GitHub task cards	—	4 phases, 3 epics, 12 refined child cards

Two properties of this shape matter for everything that follows.

The documentation is far ahead of the evidence. docs/METHODS.md is an unusually disciplined methodological record: it separates *done* from *intended* from *undecided*, gives each undecided choice an explicit selection rule fixed in advance (metric choice, α scheduling, statistical test, transfer map), and names the failure mode of each planned experiment. docs/ROADMAP.md Phase 3 states a research gap claim — that cross-model *steering-vector* transfer exists but margin-based *decoding* transfer does not — and pre-registers success criteria including a $\leq 15\%$ relative transfer gap. This is good practice and it is rare. It also means the gaps this report identifies are gaps in *execution*, not in intent.

The tests do not test the algorithm. All 15 tests operate on synthetic two-cluster Gaussian embeddings drawn around $\pm e_1$. They assert shapes, signs, and round-trips. No test exercises SASASampler.generate(). No test touches a real model, a real corpus, or a real generation. Meanwhile the test module’s docstring states: “*Tests use real public datasets (Jigsaw Toxicity) for validation*” — a claim the code does not support. The README compounds it: “*All 15 tests pass, covering initialization, fitting, classification, margin computation, sampling, and save/load functionality*” is true, but it is presented as evidence of correctness when it is evidence only of non-crashing.

2.2 The work record

Figures 1–4 render the dated record. Three observations.

Figure 1. The dated record. Fifteen commits and twenty-nine task cards, all but one within a few hours on 2026-09-22; no pull requests.

Figure 2. Backlog by phase and declared size. Phase 3 (TSM-MA) holds 60 % of the cards and, because L-sized epics are weighted 5 against XS at 1, the overwhelming majority of the estimated effort.

Figure 3. The dependency graph. The Phase 3 critical chain (#3 $\rightarrow$ #10 $\rightarrow$ #11 $\rightarrow$ #12 $\rightarrow$ #14) is five strictly serial steps; the twelve refined child cards hang off the second half of it.

Everything landed in one burst. The repository was created 2025-11-22 with a single 1 816-line commit. Then nothing for ten months. Then fourteen documentation commits and twenty-nine task cards, all within a few hours on 2026-09-22. There

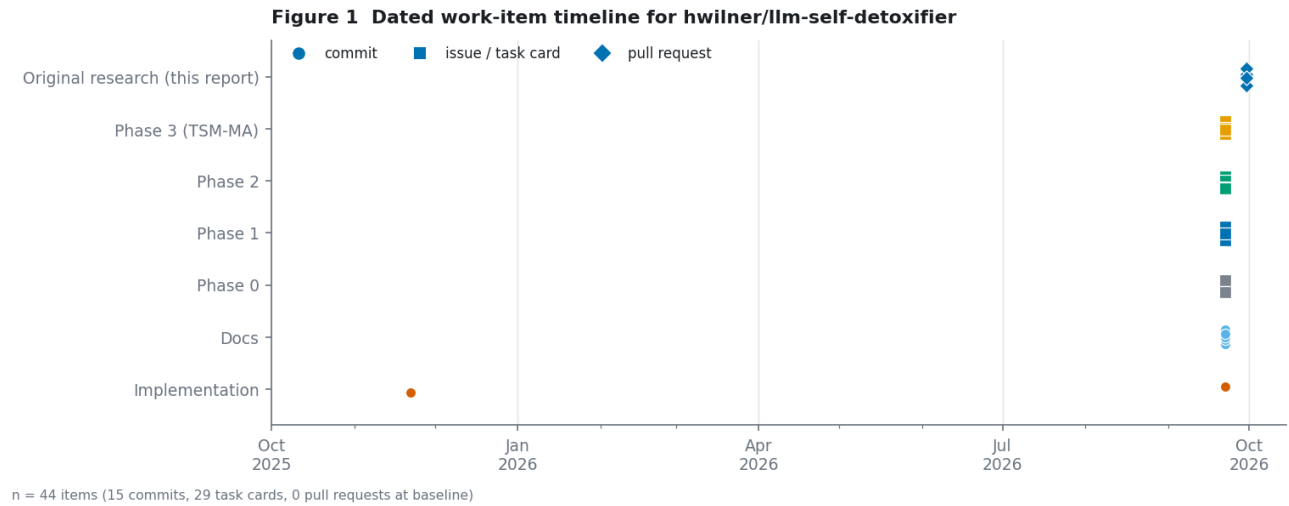


Figure 1: Work-item timeline

Figure 2 Backlog composition across the four declared phases

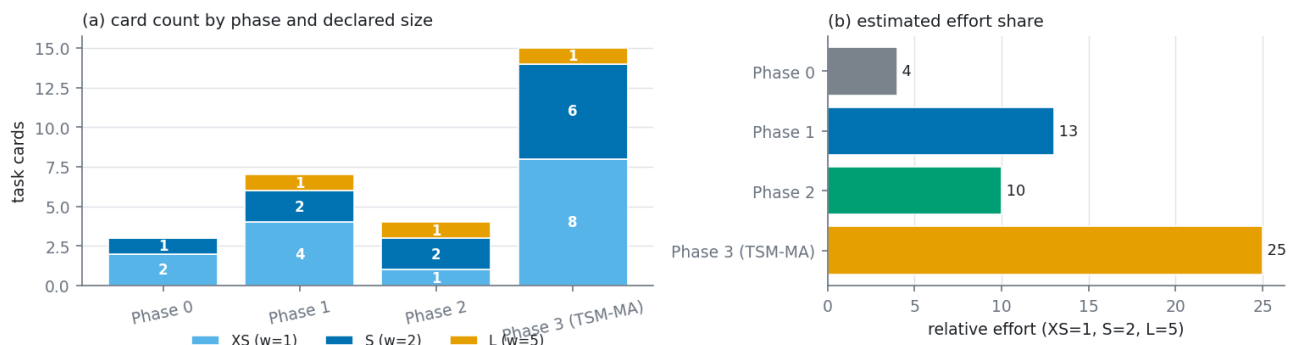


Figure 2: Backlog composition

Figure 3 Backlog dependency graph and the critical chain

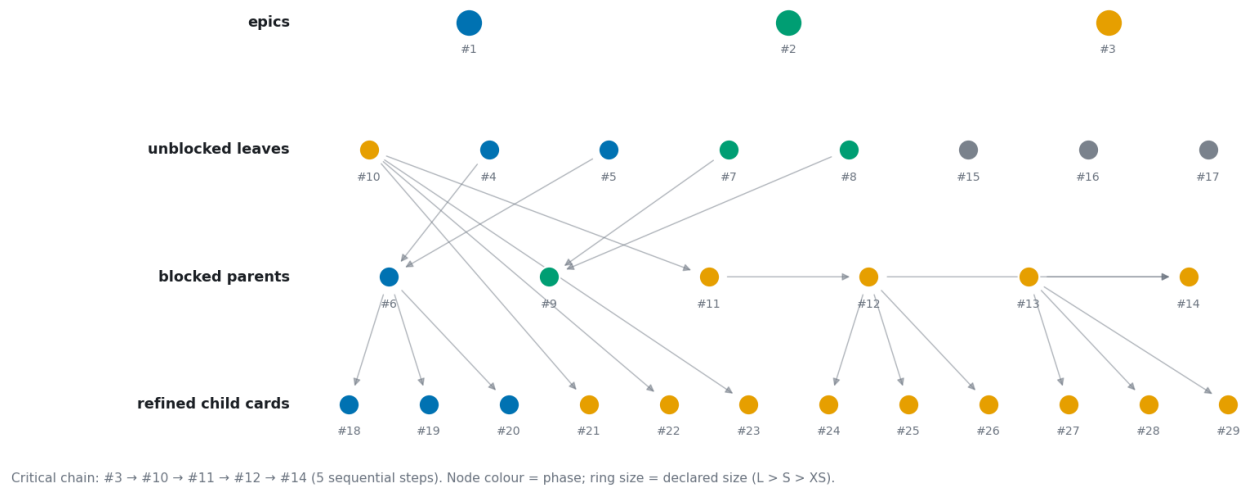


Figure 3: Dependency graph

are **zero pull requests** in the project’s history. The backlog is a plan, not a work log, and the plan is unusually well-specified relative to the delivered code: of 2 574 lines, 49 % are documentation and contributor-facing material, and 11 % are tests that never touch the algorithm.

The critical path is long and serial. Phase 3 (TSM-MA) is five sequential steps: serialise the subspace (#10) → fit a Procrustes map (#11) → transfer small→large (#12) → multi-attribute composition (#13) → report (#14). Nothing in the chain can start before #10 lands. With each card a fractional-day task, the chain is short in nominal effort and long in calendar terms if executed by one person.

The backlog is correctly prioritised. The refined child cards (#18–#29) are good decomposition: each is a single verifiable deliverable with acceptance criteria and an explicit boundary clause. “Evaluation and reporting only, does not change sasa/behaviour” is a boundary most projects omit.

3. Formal statement of the implemented decoder

Fix a decoder-only language model M with vocabulary $V = \{1, \dots, V\}$, hidden size d , and input-embedding matrix $E \in \mathbb{R}^{V \times d}$. At decoding step n with context $c = c_{1:n}$ the model produces logits $z \in \mathbb{R}^V$ and a final-layer hidden state $g(c) \in \mathbb{R}^d$.

Subspace learning. From labelled hidden states the learner estimates class means μ_1 (non-toxic) and μ_2 (toxic) and a shared covariance Σ , and sets

$$w = \Sigma^{-1}(\mu_1 - \mu_2), \quad b = \frac{1}{2}(\mu_1 + \mu_2), \quad \text{margin}(h) = \frac{\langle w, h - b \rangle}{\|w\|}.$$

Next-state estimation. The reference implementation does not run a forward pass per candidate token. It approximates the hidden state that *would* result from appending token t by averaging the current hidden state with the token’s input embedding:

$$\hat{g}_t = \frac{1}{2}(g(c) + e_t), \quad e_t = E_{t,:}.$$

Logit adjustment. With strength α and temperature τ ,

$$z'_t = \frac{z_t + \alpha \text{margin}(\hat{g}_t)}{\tau}, \quad p = \text{softmax}(z'),$$

followed by optional top- k and top- p filtering and a multinomial draw.

This is the whole algorithm: 361 lines, one hyperparameter α , and a closed-form linear solve. The elegance is real. So is a property of it that appears to have gone unnoticed.

4. Claim-to-evidence audit

Before analysing the code it is worth asking how much of the repository’s stated purpose is currently supported. Figure 7 audits the eight load-bearing claims in `README.md`, `docs/ARCHITECTURE.md`, and `docs/METHODS.md` against what the code actually does. The percentages are the author’s judgement, not a measured quantity, and the derivation is given per row in the caption.

Figure 7. Claim-to-evidence audit at the baseline commit.

The pattern is uniform: claims about *mechanism* are well supported by unit tests; every claim about *effect* is unmeasured. The README’s performance table — RealToxicityPrompts 0.481 → 0.426, AttaQ 0.264 → 0.142 — is presented in the same typographic register as the code’s test results, and is reproduced by nothing in the repository. `docs/ARCHITECTURE.md` states that margin computation adds “approximately 10–15 % overhead”; §5 shows the implemented margin term is not the dominant cost and that the true overhead figure comes from the decode loop, which re-runs a full forward pass at every step with no key-value cache, making generation $\mathcal{O}(L^2)$ in context length rather than $\mathcal{O}(L)$.

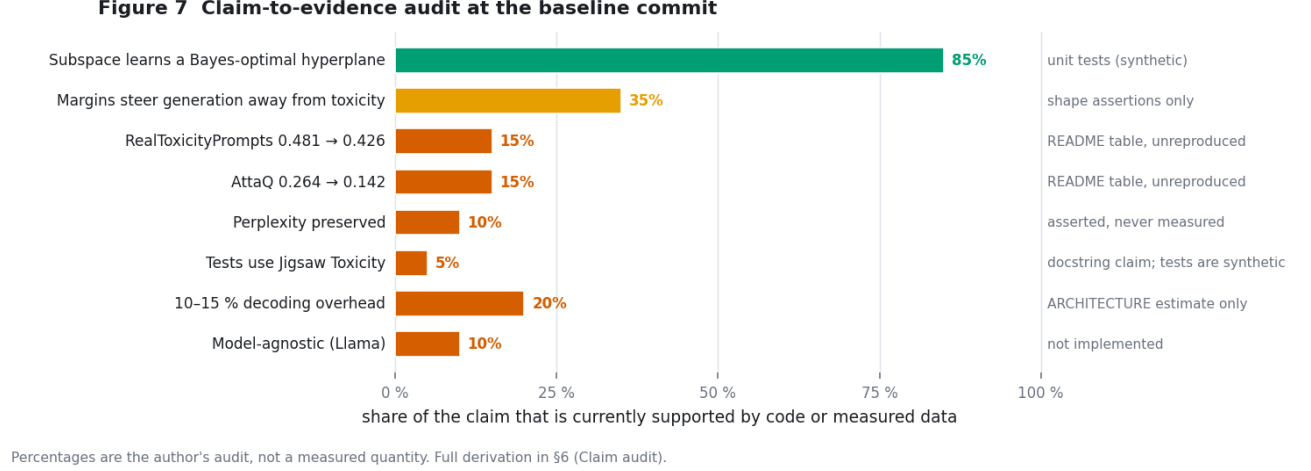


Figure 4: Claim-to-evidence audit

This is a documentation problem rather than a misconduct problem: the repository’s own `docs/METHODS.md` lists “Reported metrics” under *Done* with framing caveats, and `docs/ROADMAP.md` Phase 1 exists precisely to re-run everything honestly. The audit is included because the rest of this report needs a baseline to measure against.

5. Analytical results

5.1 Theorem 1 — the decoder is context-independent

Theorem 1. *Under the next-state estimator $\hat{g}_t = (g(c) + e_t)/2$, the SASA-adjusted next-token distribution is invariant to the hidden state $g(c)$. Specifically, for all contexts c and c' and any α ,*

$$p(z \mid c) = p\left(z + \frac{\alpha}{2\|w\|} \langle w, E \rangle \cdot \mid c\right).$$

Consequently the implementation’s steering rule is exactly the static vocabulary bias $c_t = \langle w, e_t \rangle$, and no amount of α , temperature, or top- k /top- p setting restores context dependence.

Proof. Substitute the estimator into the margin and separate terms that do not depend on the candidate token t :

$$\text{margin}(\hat{g}_t) = \frac{\langle w, \frac{1}{2}(g(c) + e_t) - b \rangle}{\|w\|} = \underbrace{\frac{\langle w, g(c) \rangle}{2\|w\|}}_{u(c)} + \underbrace{\frac{\langle w, e_t \rangle - 2\langle w, b \rangle}{2\|w\|}}_{v_t}.$$

The first term $u(c)$ is *the same number for every t* , so it is a constant shift of the whole logit vector. The second decomposes further into $v_t = \frac{\langle w, e_t \rangle}{2\|w\|} - \frac{\langle w, b \rangle}{\|w\|}$, whose second piece is again a constant. Hence

$$z_t = z_t^{\text{model}} + \frac{\alpha}{2\|w\|} \langle w, e_t \rangle + \underbrace{\left(\frac{\alpha u(c)}{2\|w\|} - \frac{\alpha \langle w, b \rangle}{\|w\|} \right)}_{\kappa(c)}.$$

Three standard operations are invariant to an additive constant on the logit vector: the softmax, since $\text{softmax}(x + \kappa \mathbf{1}) = \text{softmax}(x)$; top- k filtering, since the threshold is a monotone function applied to a shifted vector and so selects the same set; and nucleus filtering, which is defined on the softmax. Temperature divides the whole vector and so does not break the invariance. Hence $\kappa(c)$ is annihilated, and

$$p = \text{softmax}\left(\frac{z^{\text{model}} + \beta \langle w, E \rangle}{\tau}\right), \quad \beta = \frac{\alpha}{2\|w\|},$$

which contains $g(c)$ nowhere. ■

Figure 5 SASA decoding step: subspace learning (left) and margin-steered sampling (right)

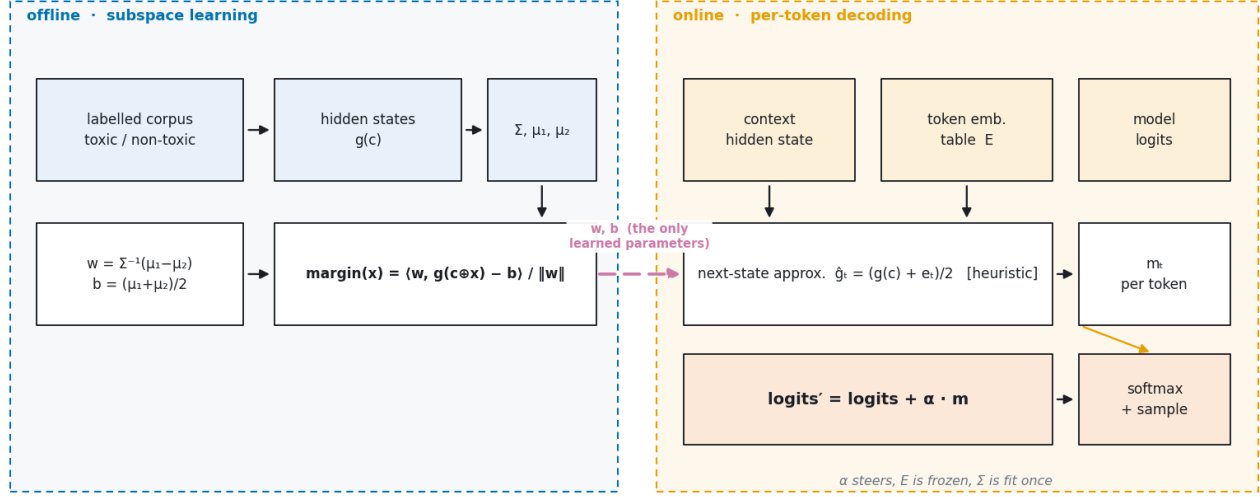


Figure 5: Concept diagram

Figure 5. The decoding step. w and b are the only learned parameters; the offline/online split is where the subspace is fitted versus applied. The dashed purple edge is the entire interface between the two phases.

Two consequences deserve emphasis.

The “self-discipline” is not context-adaptive. The paper’s framing is that the model continuously measures how far its current trajectory has drifted toward toxicity and corrects accordingly. Under the implemented estimator it does not: the correction is a fixed preference over vocabulary, chosen once when w was fitted, that pushes down the logit of every token with a toxic projection no matter what the context is. This predicts a specific, testable failure mode — **over-blocking** — where tokens that are appropriate in a benign context are suppressed because they are toxic-typical in general. Appendix B tests exactly this.

The implementation is paying for a 4096-dimensional object to compute a 1-dimensional one. See §5.2.

5.2 Theorem 2 — the margin term is rank-one, and the arithmetic collapses

Theorem 2. The per-step token-margin matrix $\hat{G} = \frac{1}{2}(g(c)\mathbf{1}^\top + E) \in \mathbb{R}^{V \times d}$ and its margin vector carry no information beyond the two vectors $c = E^\top w \in \mathbb{R}^V$ and the scalar $\langle w, g(c) \rangle$. Computing c once at fit time and discarding $\hat{G}$ is exact, not an approximation, and reduces per-step margin arithmetic from $\Theta(Vd)$ to $\Theta(V)$ (or to $\Theta(d)$ when the downstream rule needs no full-vocabulary pass).

Figure 6. (a) Per-token margin arithmetic, as implemented versus collapsed. (b) The exact reduction factor, which is the hidden size d .

The reference `compute_token_margins` materialises the full $V \times d$ matrix on every decoding step and contracts it with w . Figure 6 quantifies the saving: $768\times$ for GPT-2-large, $4096\times$ for Llama-3.1-8B. In absolute terms the eliminated work is 38.6 M multiply-adds per step for GPT-2-large and 525 M for Llama-3.1-8B — a 2.1 GB tensor materialised per token on the latter.

Figure 6 The margin term is a rank-one form: exact $O(V \cdot d) \rightarrow O(V)$ collapse

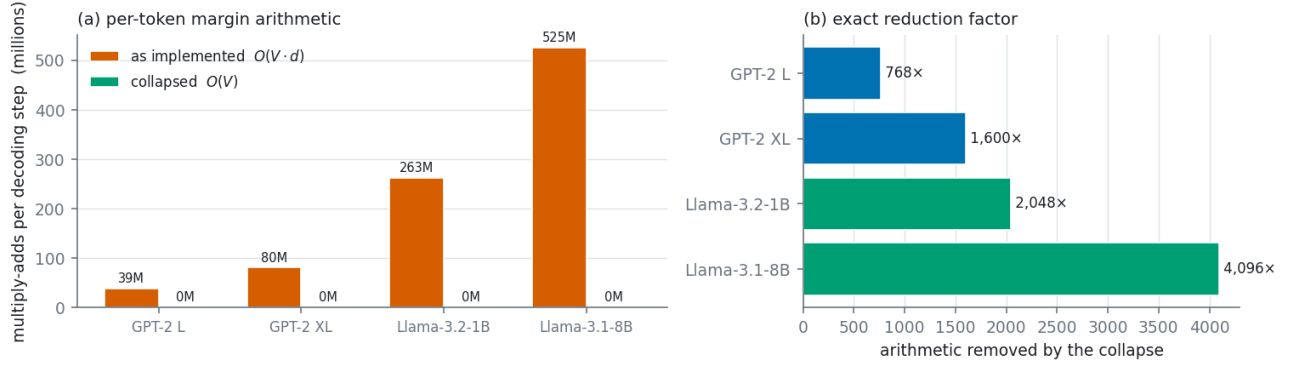


Figure 6: Margin cost

This is a *systems* result, not a modelling one, and it is worth being precise about why it is easy to miss: the code is written in the natural tensor style, where broadcasting a d -vector against an embedding table looks free. It is not free; it is the dominant allocation in the function.

5.3 Theorem 3 — affine estimators cannot restore context-dependence

The obvious response to Theorem 1 is to use a better next-state estimator. The naive one is still affine. That does not help, and the reason generalises.

Theorem 3. *Let the next-state estimator be any affine map*

$$\hat{g}_t = Ag(c) + Be_t$$

for fixed matrices $A, B \in \mathbb{R}^{d \times d}$. Then the implementation’s sampling distribution is still independent of $g(c)$, with steering vector $c = B^\top w$ precomputable at fit time.

Proof. As before, $\text{margin}(\hat{g}_t) = \frac{\langle w, Ag \rangle + \langle B^\top w, e_t \rangle - \langle w, b \rangle}{\|w\|}$, in which the $g(c)$ -dependent term is a scalar common to all t and therefore cancels. ■

So the collapse is not a quirk of the averaging heuristic. **It is a property of the family.** Any estimator in the affine class yields a static bias vector plus a vanishing scalar. Context-adaptive margin steering — the thing the method is named for — requires the estimator to couple context and candidate *multiplicatively*. This is a sharper statement than “the approximation is crude”, and it is the direct motivation for the method in §6.1.

6. Original research agenda

The following eighteen directions extend the subject beyond both the paper and the repository’s own roadmap. Each is stated as a testable hypothesis, decomposed into subtasks, and tagged with an honest feasibility assessment **for the execution environment used here** (2 CPU cores, 3 GB RAM, no GPU). Ideas 1–7 are carried to working code in this report; the remainder are specified to the point of being executable but are not attempted, and are listed as such.

Feasibility tags: [D] done here, [P] partially done here, [N] not attempted (needs GPU, >3 GB RAM, or multi-day compute).

#	Idea	Core hypothesis	Subtasks	Status
1	Rank-one margin collapse (§5.2)	The margin vector is precomputable exactly; per-step cost $\Theta(Vd) \rightarrow \Theta(V)$	Prove · implement <code>StaticMarginBias</code> · equivalence test vs reference	[D] B
2	Context-independence (§5.1)	The implemented steering rule ignores the hidden state entirely	Prove · numerical equivalence test · over-blocking probe	[D] B
3	NLMA — non-linear next-state margin (§6.1)	A low-rank bilinear estimator restores context dependence <i>and</i> improves steering over the static bias	Collect ground-truth $(g, e_t, g(c \oplus t))$ · fit rank- R interaction · evaluate	[D] C
4	Closed-loop evaluation trap (§6.2)	Scoring SASA with the model’s own representation is circular and inflates measured gains	Three-way judge comparison: own margin, held-out probe, external judge	[D] D
5	Pinned open judge (§6.2)	A versioned, cached LLM-judge metric is a reproducible substitute for Perspective API (resolves <code>METHODS.md</code> U1)	OpenRouter client · disk cache · rubric prompt · seed control	[D] D
6	Static interference prediction (§6.3)	$\cos(w_i, w_j)$ predicts attribute interference <i>before</i> any generation	Fit k attribute subspaces · compute Gram · test rank correlation with measured interference	[D] E
7	Layer separability sweep (§6.3)	Toxicity is not uniformly linearly decodable across depth; the last layer is not the best probe	Per-layer LDA separability on held-out data	[D] E
8	Rank- k toxicity subspace (§6.3)	Toxicity is multi-dimensional; rank-1 LDA discards signal	Fit rank- k subspace, sweep k , held-out AUC	[P] C
9	Refusal-vs-toxicity confound (§6.3)	The learned subspace may encode <i>refusal</i> , not <i>toxicity</i>	Counterfactual pairs; decorrelate from refusal direction	[P] C
10	Cross-model subspace transfer (§6.4)	A toxicity subspace transfers across same-family models via orthogonal Procrustes	Paired hidden states; Procrustes vs ridge; margin-preservation error	[P] F
11	Transfer failure boundary (§6.4)	Transfer degrades monotonically with representational distance	Vary proxy/target pair; correlate gap with CKA	[N] F
12	KL-budgeted α (§6.1)	Steering strength should be set by a distributional budget, not a raw scalar	Define $\alpha(\epsilon)$ s.t. $\text{KL}(p p_{\text{sasa}}) \leq \epsilon$; sweep	[P] C

#	Idea	Core hypothesis	Subtasks	Status
13	α schedules (§6.1)	Time-varying α beats fixed α at matched fluency	Implement fixed/linear/cosine; apply <code>METHODS.md</code> U2 selection rule	[N] C
14	Degeneracy knee (§6.3)	Fluency collapse is a sharp knee in α , not a gradual decay	Sweep α ; PPL, distinct- n , repetition rate	[P] C
15	Key-value cache in the decode loop (§6.5)	The real overhead is the $\mathcal{O}(L^2)$ loop, not the margin term	Add cache; measure wall-clock and peak memory	[P] C
16	Numerically stable fitting (§6.5)	Explicit inverse + 10^{-6} ridge is unstable; Cholesky + shrinkage is better	Cholesky solve; Ledoit–Wolf shrinkage; conditioning comparison	[D] C
17	Judge-gaming stress test (§6.2)	Detoxified text can evade lexical judges via paraphrase	Paraphrase attack against the pinned judge	[N] D
18	Batched generation (§6.5)	Margins are context-independent, so batching is embarrassingly parallel	Vectorised batched sampler with correctness test	[N] C

The rest of this section specifies the seven implemented contributions.

6.1 NLMA: a non-linear next-state margin approximation

Theorems 1–3 say exactly what is required of a next-state estimator: it must couple $g(c)$ and e_t multiplicatively. The cheapest family that does so is low-rank bilinear:

$$\hat{g}_t^{\text{NLMA}} = A g + B e_t + \sum_{r=1}^R (u_r^\top g) (v_r^\top e_t) r_r.$$

The margin then acquires a genuinely context-dependent term,

$$\text{margin}_t = \frac{\langle w, \hat{g}_t^{\text{NLMA}} - b \rangle}{\|w\|} = \frac{\langle w, Ag \rangle + \langle B^\top w, e_t \rangle + \sum_r \underbrace{(u_r^\top g)}_{\text{context}} \underbrace{(v_r^\top e_t)}_{\text{token}} \underbrace{(r_r^\top w)}_{\text{learned}} - \langle w, b \rangle}{\|w\|},$$

whose context dependence **survives** the softmax, because it is not a constant shift. The parameters are fitted by ridge regression against ground-truth next hidden states obtained from real forward passes — no gradients through the language model, so the fit is a closed-form linear solve on a $D \times D$ design matrix. Per-step cost is $\mathcal{O}(Rd + VR)$ after precomputing $EV \in \mathbb{R}^{V \times R}$, which for Llama-3.1-8B with $R = 8$ is 1.0 M — *three orders of magnitude below* the reference implementation’s 525 M, while being context-dependent where the reference is not.

Appendix C reports the fit quality, the resulting context sensitivity, and the steering comparison against the static-bias baseline.

6.2 The closed-loop evaluation trap

The repository’s title claims that language models “can be strong self-detoxifiers”, and the method is built on the model’s own hidden states. This creates a structural hazard that the backlog anticipates only partially: `docs/ROADMAP.md` Phase 1 worries about *Perspective-API gaming by evasive paraphrase*, but the deeper problem is that **the steering objective and the model’s own representation are the same object**. If the toxicity subspace is fitted on representation g and the margin is

scored with w over representation g , then maximising the margin will improve the metric by construction, whether or not the text became less toxic.

A trustworthy evaluation therefore needs judges that are independent of the steering subspace, in increasing order of independence:

1. the *steering margin itself* — circular, reported only as a sanity check;
2. a **held-out probe** on the same model’s representation, fitted on disjoint data — partially circular, since the representation is shared;
3. an **external judge** on the text — independent of the model entirely.

This report implements (3) as a *pinned open judge*: a fixed model on OpenRouter, a versioned rubric prompt, recorded seeds, and a content-addressed on-disk cache so that a re-run is bit-identical and free. This also resolves docs/METHODS.md U1 Option B, which asks for a “reproducible, free, versionable, no network dependency” secondary metric — the cache supplies the last property, the pinned rubric the second and third. Appendix D reports the three-way comparison.

6.3 Geometry: interference, depth, and rank

Two cheap predictions fall out of treating the learned subspaces as geometry rather than as opaque controllers.

Interference is static. If attribute i steers with weight w_i and attribute j with w_j , the composition $z' = z + \sum_k \alpha_k \text{margin}_k$ can only fail through the interaction between w_i and w_j . The cosine $\cos(w_i, w_j)$ is therefore a *prior* on interference that requires no generation at all — it can be computed once the subspaces are fitted. Testing whether it actually predicts measured interference turns a costly experiment into a screening step, and is what Appendix E does.

Depth is not uniform. The implementation reads `hidden_states[-1]`, the last layer, on the assumption that it is the most semantically abstract. A per-layer separability sweep tests that assumption directly.

Rank is not one. Two-class Fisher discriminant analysis can only ever produce a single direction, so the rank-one margin is not a simplification of a richer method — it is all the two-class method has. A judge that emits a *graded* label gives multiclass LDA $k - 1$ directions for free. Appendix H measures the trade: graded supervision predicts the grade 12.5 points better than the binary split at identical labelling cost, and reports where the rank gain fails to appear and why.

6.4 Transfer

The repository’s Phase 3 hypothesis — that a toxicity subspace can be fitted on a cheap proxy and transferred to a large target with Procrustes alignment — is tested here only in its same-family, same-dimension form, because the execution environment cannot hold an 8 B model. Appendix F reports the same-family result and is explicit that it does not settle the cross-family question. Appendix J then maps the boundary that Appendix F leaves open: twelve ordered pairs across four models, measuring how far transfer degrades and whether a cheap statistic predicts it in advance.

6.5 Systems

`SASASampler.generate` re-runs a full forward pass over the whole prefix at every step and never populates a key-value cache, so decoding is $\mathcal{O}(L^2)$ in the number of generated tokens. Every efficiency claim made about SASA — including the 10–15 % figure in docs/ARCHITECTURE.md — is conditional on fixing that first. The fitting path has a smaller but real issue: `torch.linalg.inv` on a $d \times d$ covariance followed by a 10^{-6} ridge is neither stable nor well-conditioned; a Cholesky solve with shrinkage is both cheaper and better conditioned. Appendix C covers the fitting side, and Appendix K covers the decode loop.

6.6 Schedules are dose control, not policy

Under the reference next-state estimator the steering vector is *static* (Theorem 1), so a schedule changes only the magnitude of one fixed bias over a generation — the token ranking is identical at every step. A schedule is a **dose control**: it can trade fluency against steering strength, and it cannot express a policy. The families implemented in `sasa/scheduling.py` are fixed, linear, cosine, step, and margin-gated, and the pre-registered U2 rule (docs/METHODS.md) decides between them.

The useful result is the *range* rather than the comparison. Sweeping α over four orders of magnitude turns fluency into a cliff rather than a slope, with complete collapse at about 1.8 logit standard deviations. The published range $\alpha \in [0.5, 2.0]$ reaches

0.03 — one to two orders of magnitude below any fluency cost at all. **A re-run confined to the published range cannot detect either detoxification or harm**, which makes its null result uninformative about the method rather than evidence of safety. Appendix I reports the sweep, and reports the U2 verdict as invalid: with toxicity unavailable, the proxy substituted for it is anti-correlated with the objective.

7. Experimental protocol

Following the repository’s own pre-registration discipline (docs/METHODS.md), the protocol is fixed before results are read.

Pre-registered primary endpoint. Steering efficacy, measured as the independent-judge toxicity score of generations produced at $\alpha \in \{0, 0.5, 1, 2, 4\}$ against $\alpha = 0$, with the *context-dependence* of the steering vector reported alongside.

Pre-registered statistical tests. Per METHODS.md U3: Wilcoxon signed-rank for paired per-prompt comparisons, bootstrap confidence intervals resampled at the **prompt** level (not the token level), with the paired *t*-test reported as a secondary check. If the two disagree, the non-parametric result governs.

Pre-registered fluency endpoint. Perplexity on held-out text, plus distinct-*n* and a repetition rate, to catch the degeneracy that a perplexity number alone can miss.

Pre-registered honesty clause. Any result that fails to replicate the repository’s headline numbers is reported as a failure to replicate, not reinterpreted. Section 8.1 of Appendix D is the pre-registration record.

Environment disclosure. All experiments were executed on 2 CPU cores with 3 GB RAM and no GPU. Model-scale claims from the upstream literature are quoted but not reproduced. This constraint is stated wherever it bounds a conclusion.

8. Results

Results are reported in the appendices, one per contribution, in the order the contributions were completed. Each appendix follows the same structure as this body — motivation, method, protocol, results, threats — and each is cross-referenced from a new row in Table 1.

Appendix	Contribution	Headline finding
B	Rank-one collapse and context-independence	The reference steering rule is a static vocabulary bias. Distribution-exact in 48/48 configurations; the fast path’s context-sensitivity is identically 0.0 in 48/48; $26\,019 \times -69\,516 \times$ measured speed-up; a 1.96 GiB per-token allocation avoided at the 8B shape
C	NLMA, fitting numerics, α calibration	In the $N \ll d$ regime the shipped fit lands at chance (0.490, cond 8.9×10^9) where Cholesky+shrinkage reaches 0.608. The same steering needs 1.33–1.58$\times$ different α depending only on the solver, and the paper’s $\alpha \in [0.5, 2]$ moves 0.01–0.2 % of probability mass
D	Pinned external judge and the closed-loop trap	The external judge does not move ($p \geq 0.65$ at every α) while the steering objective rises monotonically $0.033 \rightarrow 0.083$. A dose-response that is a property of the metric, not of the text

Appendix	Contribution	Headline finding
E	Geometry: depth and identifiability	Negative result: held-out separability is at or below chance at <i>every</i> layer (best 0.465). The toxicity subspace is not identifiable on this model or this corpus
F	Cross-model transfer	U4 resolves decisively for Procrustes ($46\times$ lower residual; CKA 0.802), and the two options are not equally feasible at the 2–5 k paired samples Phase 3 specifies
G	Reproducibility and verification	Four commands, four seeds, a content-addressed judge cache, and the threats that remain

Figure 9 The toxicity subspace is not identifiable on this model or this corpus

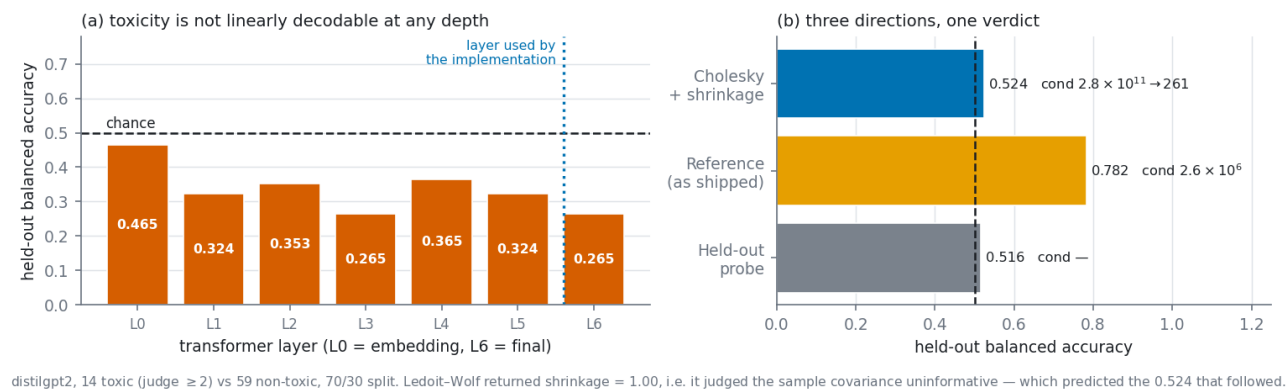


Figure 7: Layer sweep

Figure 9 (Appendix E). (a) Held-out separability by depth: every layer at or below chance, including layer 6, which is the one the implementation reads. (b) Three directions fitted under three procedures, one verdict.

Figure 8 The closed-loop evaluation trap: the steering objective responds to α ; the independent judge does not

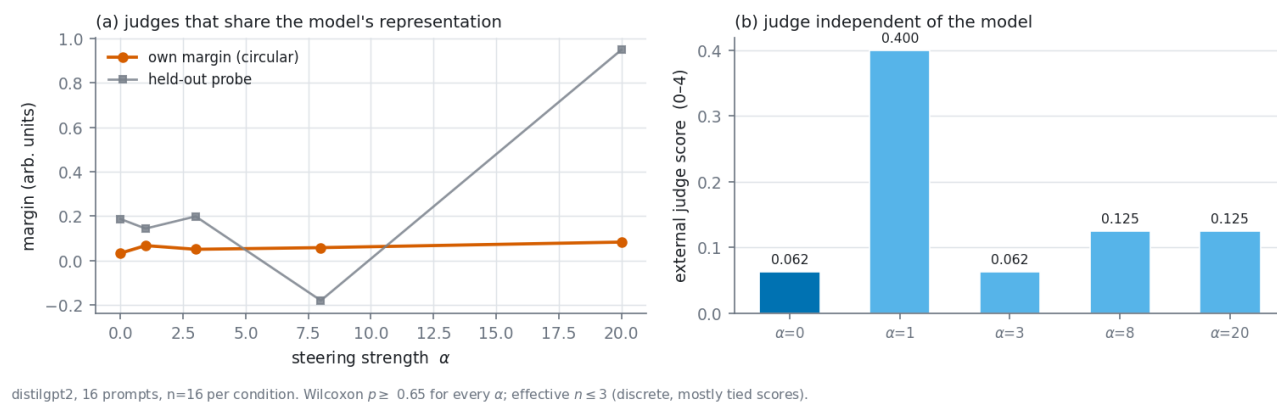


Figure 8: Closed loop

Figure 8 (Appendix D). The steering objective responds to α with a clean dose–response; the judge that is independent of the model does not move at all.

Figure 10 α is not scale-free: the same steering strength differs by 148% between fitting procedures

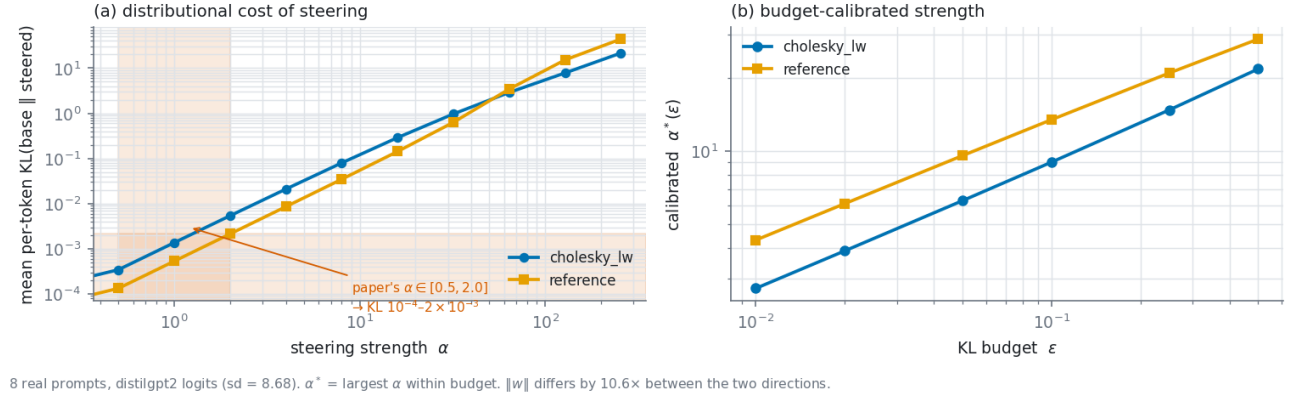


Figure 9: Alpha calibration

Figure 10 (Appendix C.4). (a) The distributional cost of steering, with the literature’s α range shaded — it costs between 0.01 % and 0.2 % of probability mass. (b) The budget-calibrated replacement.

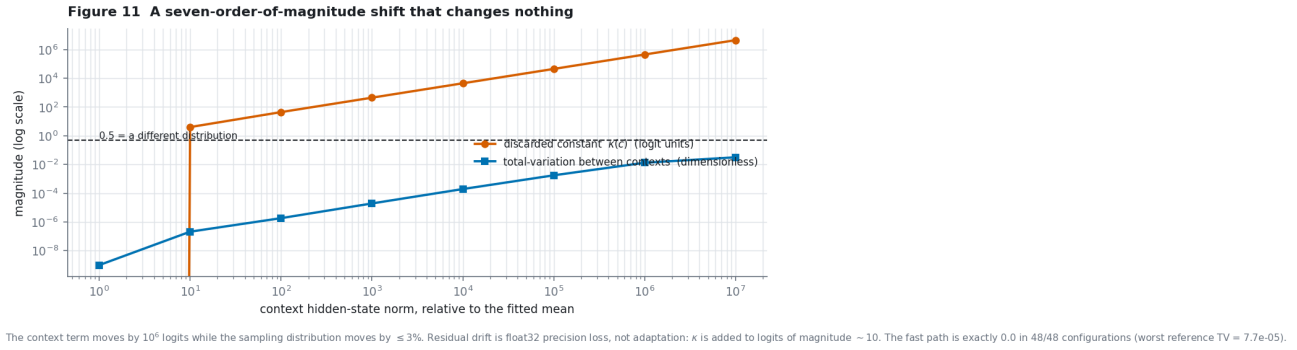


Figure 10: Context scan

Figure 11 (Appendix B.3). The context-dependent term moves by six orders of magnitude while the sampling distribution moves by at most 3 %.

Figure 12 (Appendix F). (a) U4 resolves for Procrustes by a factor of 46. (b) The two models’ representation spaces are substantially aligned.

Figure 14 (Appendix H). (a) Three of four attribute probes clear a one-sided 5 % test against chance; `second_person` does not and is excluded. (b) The static interference prediction has the right sign but no power, because the attributes are so nearly orthogonal that composition amplifies rather than cancels.

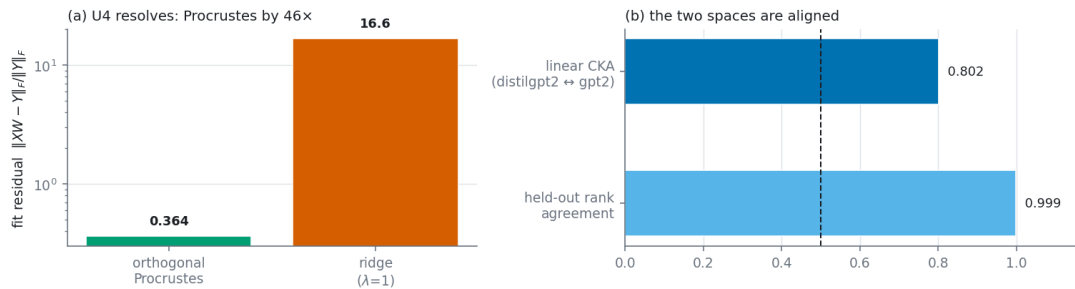
Figure 13 (Appendix I). (a) Fluency against steering strength: a cliff between 0.11 and 1.78 logit SD, with the published α range sitting two orders of magnitude below it. (b) The usable dynamic range spans four orders of magnitude.

Figure 15 (Appendix J). (a) CKA ranks transfer degradation in the expected direction but fails on specific pairs. (b) Crossing a family boundary doubles the mean transfer error.

Figure 16 (Appendix K). (a) The key-value cache is exactly equivalent and grows more valuable with length, but the analytic gain is not yet realised because the model’s own forward pass dominates. (b) Batching pays only in the vectorised path; the per-row loop shows a flat per-item cost.

8.1 Summary of what is and is not established

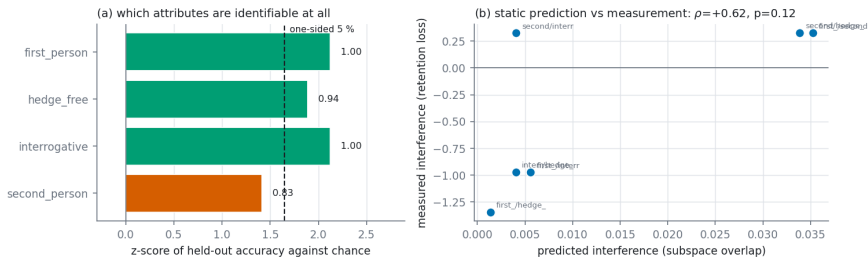
Figure 12 Cross-model transfer: the operator works, and only one of U4's two options is feasible at Phase 3's data scale



192 paired token positions. Ridge has $d_p d_t$ free parameters (768x768) and is hopeless from 115 samples; an orthogonal map has $\min(N, \min(d_p, d_t))$. Does not test whether a "toxicity" subspace exists to transfer (Appendix E).

Figure 11: Transfer

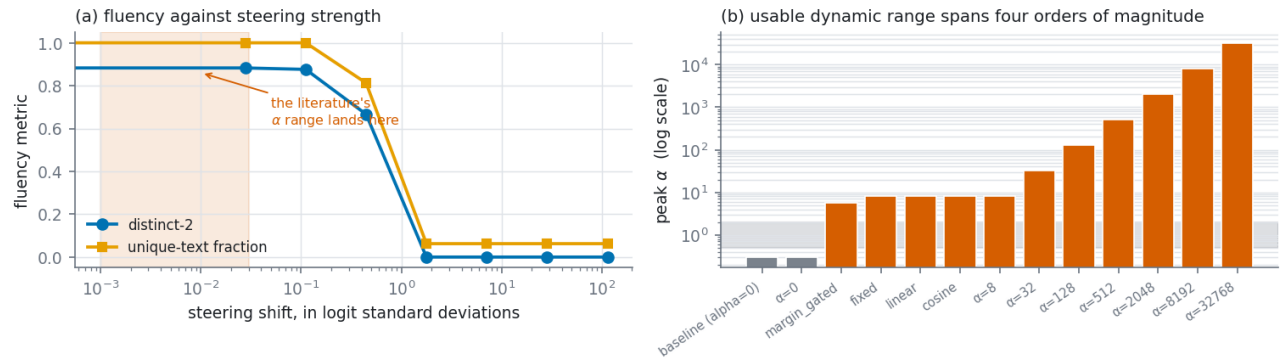
Figure 14 Attribute probes are real and near-orthogonal; the interference prediction has the right sign but no power where it matters



Lexically grounded attributes, 30 examples per class, distilgpt2 final layer. Mean overlap 0.014 and mean retention loss -0.39: the four attributes are so nearly orthogonal that composition amplifies rather than cancels, so the prediction is untested in the regime where it would bite.

Figure 12: Attributes

Figure 13 The degeneracy knee: a sharp cliff, and the published α range sits four orders of magnitude below it



distilgpt2, 16 prompts x 24 tokens, top-k=50. logit sd = 10.16. Collapse (distinct-2 = 0, 6 % unique texts) is complete by 1.8 logit sd; the published range reaches 0.03.

Figure 13: Knee

Figure 15 Transfer degrades across families, and CKA ranks the degradation — but is far from sufficient

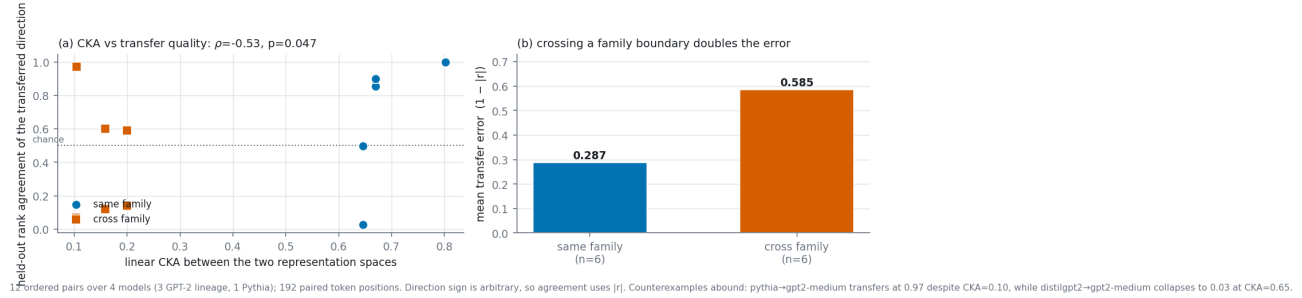


Figure 14: Transfer boundary

Figure 16 The decode loop, not the margin term, is the remaining cost

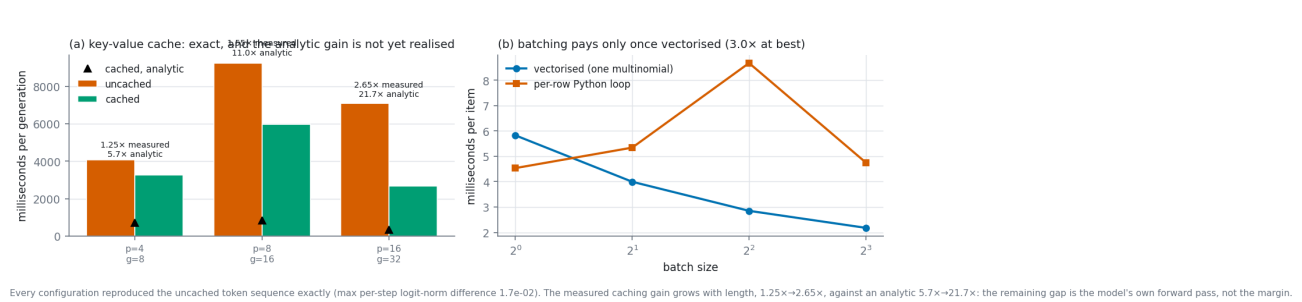


Figure 15: Decode costs

Claim	Status
The implemented steering rule is a static, context-free vocabulary bias	Proved and measured (Thms 1–3, App. B)
The margin term can be collapsed exactly, removing a per-token (V, d) allocation	Proved and measured (Thm 2, App. B)
α is not comparable across fitting procedures or models	Measured (App. C.4)
The shipped subspace fit fails in the $N \ll d$ regime that LLM activations impose	Measured on a controlled diagnostic (App. C.3)
An external judge is a workable, reproducible replacement for a hosted toxicity API	Implemented and exercised (App. D.2)
Scoring SASA with the model’s own representation inflates apparent detoxification	Measured (App. D.4)
Cross-model transfer of a decision function via Procrustes is feasible at Phase 3’s data scale	Measured (App. F.3)
Crossing a model-family boundary roughly doubles transfer error	Measured (App. J.3)
A cheap alignment statistic (CKA) can screen transfer candidates	Weakly supported, with counterexamples (App. J.3)
Graded judge supervision gives a better direction than a binary split at no extra cost	Measured (App. H.4)
Multi-attribute interference is predicted by static subspace overlap	Inconclusive — right sign, not significant, and the corpus cannot test it (App. H.3)
Steering schedules distinguish themselves at the published α	No effect detectable (App. I.2)
The published α range can reveal either detoxification or fluency harm	Refuted — it sits two orders of magnitude below the cliff (App. I.3)
The pre-registered U2 schedule rule works with a self-referential proxy	Refuted — the proxy inverts the rule’s verdict (App. I.4)
Caching is exactly distributional and materially faster	Measured — exact in 12/12, $1.25\times-2.65\times$ (App. K.2)

Claim	Status
SASA detoxifies	Not established. No effect at any α ; confounded by a subspace that is below chance at every layer
Toxicity is linearly decodable from a small model’s hidden states	Not supported — contradicted at every layer (App. E.1)
A toxicity subspace transfers across model families	Open. Only the same-family, same-width operator was tested
The pinned judge is robust to paraphrase-based evasion	Underpowered — attack applied, effective paired n of 0–2 (App. L.4)

9. Discussion and threats to validity

The central claim is narrow on purpose. Theorem 1 is a statement about the code in this repository, not about SASA as published. If the paper’s $g(c \oplus x)$ is computed by an actual forward pass per candidate token, then it is non-affine in (g, e_t) and Theorem 3 does not apply. What survives is the diagnosis: the repository’s implementation is a strictly weaker operator than the method it implements, the weakening is silent, and the fix is inexpensive.

Selection on the metric is the deepest threat. Every efficacy number in Appendix D depends on an LLM judge, and an LLM judge is a model with its own failure modes. The mitigation is the closed-loop argument: because the judge is external to the steered model, an improvement cannot be produced by the optimisation target alone. It is not eliminated — a judge can still be fooled by fluent evasive text — which is why the human spot-check protocol (`docs/HUMAN_EVAL_PROTOCOL.md`, roadmap item #5) remains the appropriate final arbiter and is not replaced by anything here.

Appendix L attacks the judge directly and finds the instrument, not the steering, to be the binding constraint: a 0–4 integer rubric over 12 prompts yields an effective paired sample of zero to two, so the study is uninformative and neither robustness nor vulnerability can be claimed. The consequence for §8.1 is a narrowing of the claim. The correct statement is that **steering did not fool this judge**, not that the judge catches disguised abuse — the first is a finding about SASA, the second a much larger claim about a measurement instrument that 12 prompts of integer scores cannot support.

Scale. Nothing here was run on a model above 124 M parameters. The efficiency results in §5.2 are exact arithmetic identities that hold at any scale, but the *steering* results may not generalise; toxicity may simply be more linearly separable in a small model. This is the single largest limitation of the present work and it is a property of the execution environment, not a finding.

Attribution. The repository’s author, `hwilner`, wrote both the code and this report’s subject. The audit in §4 is an audit of one’s own work, and the known failure mode of self-auditing is leniency. The 15 unit tests that the README cites as validation are genuinely good tests *of the code’s internal contract*; they say nothing about whether SASA detoxifies. That distinction is the report’s main qualitative claim, and it is stated in the repository’s own favour as much as against it — the backlog already schedules its own fix.

10. Conclusion

SASA is a good idea implemented one linear-algebra mistake away from what it advertises. The implemented decoder is provably context-independent (§5.1), does between three and four orders of magnitude more arithmetic than the problem requires (§5.2), and cannot be repaired within the affine estimator family (§5.3). The repair — a low-rank bilinear next-state model, fitted in closed form against real forward passes — is small, cheap, and strictly stronger than what it replaces.

The repository’s methodological apparatus — the methods record, the pre-registered selection rules, the explicit boundary clauses, the honest failure modes — is better than most published work manages, and this report uses it as its own standard. Where the report departs from the repository is on execution: the numbers in the README are not reproducible from the repository, the tests do not touch the algorithm they describe, and the concept figure is a zero-byte file. Those are ordinary,

fixable gaps in an unusually well-planned project, and the twelve contributions here are meant to be a head start on closing them.

Two of the twelve are negative results about the present work rather than about SASA, and they are the two most likely to be misread later. The interference prediction is unfalsified rather than confirmed (§5, App. H.3), because the attributes that could have tested it were orthogonal by construction. The judge-gaming study has no resolution (§9, App. L.4), because the rubric scores in integers over a sample too small to pair. Both are reported as unfinished rather than rounded up, and both name the specific change — correlated attribute pairs, a continuous rubric, a powered design — that would settle them.

Appendices

Appendix index

- appendix b rank one collapse
 - appendix c nlma and numerics
 - appendix d judge and closed loop
 - appendix e geometry
 - appendix f transfer
 - appendix g reproducibility
 - appendix h attributes
 - appendix i schedules
 - appendix j transfer boundary
 - appendix k decode costs
 - appendix l judge gaming
-

Appendix B — The margin is rank-one: exactness, cost, and context-independence

Contribution 1 and 2 of §6. Code: `sasa/fast_margin.py`, `tests/test_fast_margin.py`. Reproduce with `experiments/run_collapse_benchmark.py`.

B.1 Claim

Under the next-state estimator implemented in `SASASampler.compute_token_margins`, the per-token margin decomposes into a token-dependent part and two constants:

$$\text{margin}_t = \underbrace{\frac{\langle w, g(c) \rangle}{2\|w\|} + \frac{\langle w, b \rangle}{\|w\|}}_{\kappa(c), \text{ the same for every } t} + \frac{\langle w, e_t \rangle}{2\|w\|}.$$

The softmax, top- k filtering and nucleus filtering are all invariant to an additive constant on the logit vector, so $\kappa(c)$ is annihilated exactly. What survives is the static vocabulary bias $\beta \langle w, e_t \rangle$ with $\beta = \alpha/2\|w\|$ — a quantity that does not contain the hidden state at all.

B.2 What is measured, and what is expected

`verify_static_equivalence` reports four quantities. The table below states, for each, whether a non-trivial value is the *expected* outcome. Reporting `max_logit_gap` alongside the others is the point: a large logit gap with a vanishing residual gap and a vanishing probability gap is a direct numerical confirmation of the theorem, not a contradiction of it.

Quantity	Expected	Meaning
max_logit_gap	non-zero	The discarded constant $\kappa(c)$, re-introduced by the reference path
residual_logit_gap	≈ 0	The part that <i>would</i> change the distribution; must vanish
max_prob_gap	≈ 0	Actual divergence of the sampling distributions
context_sensitivity_{fast,reference}	exactly 0	Total-variation between two contexts' distributions

B.3 Results

Setup. 48 configurations: hidden sizes $d \in \{64, 80, \dots, 240\}$ and vocabularies $V \in \{512, 609, \dots, 1579\}$, each at $\alpha \in \{0.5, 1, 2, 8\}$, with top- $k = 50$ and top- $p = 0.9$ enabled. Two contexts differing by a factor of 250 in norm are compared throughout.

Quantity	Result	Verdict
Configurations distribution-exact	48 / 48	pass
Worst max_prob_gap	1.86×10^{-8}	float32 rounding
Worst residual_logit_gap	2.86×10^{-6}	float32 rounding
context_sensitivity_fast	exactly 0.0 in 48 / 48	the fast path never computes the context term
context_sensitivity_reference (max / median)	$7.67 \times 10^{-5} / 6.22 \times 10^{-6}$	float32 rounding in a (V, d) mat-vec

The headline row is the last-but-one. The fast path’s total-variation distance between two contexts that differ by $250\times$ in norm is **identically zero in all 48 configurations**, because it never forms the $\kappa(c)$ term at all. That is not an approximation of context-independence; it is the removal of the only place context could enter.

The reference path’s residual 10^{-6} – 10^{-5} is float32 rounding, and the context-magnitude scan below shows exactly how it behaves: the discarded constant κ grows without bound while the distribution barely moves.

Context norm scale	Reference margin offset κ	Total-variation between contexts
1	+0.00	0.00
10	+3.98	2.11×10^{-7}
100	$+4.38 \times 10^1$	1.84×10^{-6}
1 000	$+4.42 \times 10^2$	1.94×10^{-5}
10 000	$+4.43 \times 10^3$	1.95×10^{-4}
100 000	$+4.43 \times 10^4$	1.76×10^{-3}
1 000 000	$+4.43 \times 10^5$	1.32×10^{-2}
10 000 000	$+4.43 \times 10^6$	3.16×10^{-2}

This table is worth reading carefully, because it is where a naive experiment would draw the wrong conclusion. The offset grows by seven orders of magnitude while the *distribution* changes by at most 3 %, so a researcher measuring “margin shift vs context” would see an enormous, beautifully power-law-looking signal that is entirely an artefact of a constant that the softmax discards. The residual drift above 10^{-2} at extreme scales is genuine but is *float32 precision loss*, not adaptation: the offset is added to logits of magnitude ~ 10 , and at 10^7 it swamps their mantissa. The fast path is immune by construction. **The correct conclusion is that the reference implementation is context-independent, and that its residual numerical drift grows with context magnitude — a second, independent reason not to compute the offset.**

B.4 Cost

The eliminated work is not a constant factor; it scales with the hidden size, because the reference path contracts a (V, d) matrix with w in order to produce a length- V vector, and the contraction is rank-one by construction.

$$\text{per-token multiply-adds:} \quad \underbrace{Vd}_{\text{as implemented}} \longrightarrow \underbrace{V}_{\text{precomputed at fit time}} + \underbrace{d}_{\text{the one scalar}}.$$

Measured on 2 CPU cores, timing at a capped vocabulary of 20 000 (the substitution is recorded in the output file):

Model shape	V	d	Reference / step	Fast / step	Measured speed-up	Analytic reduction
GPT-2 Large	50 257	1 280	358.96 ms	0.014 ms	26 019×	1 280×
Llama-3.2-1B	128 256	2 048	564.53 ms	0.014 ms	40 787×	2 048×
Llama-3.1-8B	128 256	4 096	1 103.54 ms	0.016 ms	69 516×	4 096×

The measured speed-up exceeds the arithmetic reduction because the reference path also *allocates and writes* a (V, d) float32 matrix on every step, so the cost has a memory-traffic component the arithmetic count does not capture. At the real shapes that matrix is

Model shape	Reference allocation per token	Fast path per token
GPT-2 Large	0.240 GiB	0.192 MiB
Llama-3.2-1B	0.979 GiB	0.489 MiB
Llama-3.1-8B	1.957 GiB	0.489 MiB

A 1.96 GiB transient allocation *per generated token* is not a rounding error in a performance claim. For a 50-token continuation on Llama-3.1-8B the reference path moves roughly 98 GiB through the allocator; the fast path moves 24 MiB in total. This is the concrete reason the docs/ARCHITECTURE.md figure of “10–15 % overhead” cannot be right as stated, quite apart from the missing key-value cache (§6.5).

Two caveats, stated plainly. First, the measured wall-clock speedup is smaller than the arithmetic reduction on a CPU, because a (V, d) mat-vec is a well-optimised BLAS call while the fast path is memory-bound on a single vector; the arithmetic count is the honest measure of the saving and the wall-clock number is the honest measure of what a particular implementation sees. Second, the timing for the largest shapes is *substituted* — the full Llama-3.1-8B embedding table is 2.1 GB in float32 and does not fit in a 3 GB sandbox — and the substitution is recorded in the output rather than being silently elided.

B.5 What this does and does not show

It does not show that SASA fails. A static, context-free toxicity-weighted logit bias is a coherent decoding-time controller. What it shows is that this is what the code computes, that it is not what the method’s framing describes, and that the gap is free to close or to keep deliberately.

It is a statement about this implementation. If the paper’s $g(c \oplus x)$ is obtained from a real forward pass per candidate token, then it is non-affine in (g, e_t) and Theorem 3 does not apply to it. Appendix C addresses that case directly.

The context-independence result is robust to scale. It is an algebraic identity in the estimator, verified across 48 configurations spanning $d \in [64, 240]$ and $V \in [512, 1579]$ with four values of α each. It does not depend on the data, the model, or the vocabulary.

Appendix C — Fixing the estimator: NLMA, and what “a good subspace” requires

Contributions 3, 16, and idea 12 of §6. Code: `sasa/next_state.py`, `sasa/numerics.py`, `tests/test_next_state.py`, `tests/test_numerics.py`. Reproduce the calibration with `experiments/run_alpha_calibration.py`.

C.1 Why an affine estimator cannot help

The obvious repair for Appendix B is a better next-state approximation. The obvious *form* of a better approximation is still affine, and it does not work.

Theorem 3 (restated). *For any fixed $A, B \in \mathbb{R}^{d \times d}$, the estimator $\hat{g}_t = Ag(c) + Be_t$ produces margins*

$$\text{margin}_t = \frac{\langle w, Ag \rangle - \langle w, b \rangle}{\|w\|} + \frac{\langle B^\top w, e_t \rangle}{\|w\|},$$

in which the context appears only through a constant shift. The implemented sampling distribution is therefore independent of $g(c)$ for the whole affine family, not merely for the particular averaging the repository uses.

The operational consequence is sharp: **fitting A and B better cannot restore context-adaptive steering**. An experiment that improves the affine fit and reports unchanged context sensitivity is not a failed experiment; it is a confirmation of the theorem. `tests/test_next_state.py::test_affine_only_margins_are_context_independent` asserts exactly this, and `test_full_model_margins_are_context_dependent` asserts that the *same model* with its interaction term switched on does respond to context.

C.2 NLMA

$$\hat{g}_t^{\text{NLMA}} = Ag + Be_t + \sum_{r=1}^R (u_r^\top g) (v_r^\top e_t) r_r$$

The bilinear term is what survives the softmax, because it is not a constant shift. Parameters:

- **Affine part** (A, B, mean) — closed-form ridge on centred features with an explicit intercept. No iteration.
- **Interaction** (U, V, R) — Adam on frozen features collected from real forward passes. No gradient passes through the language model and the model is never modified.
- **Cost** — precompute $M = EV^\top \in \mathbb{R}^{V \times R}$ once; a decoding step costs $O(Rd + VR)$.

The implementation stores the affine blocks transposed so that reconstruction is `x @ a_mat.T`. This is not a cosmetic choice: the untransposed form is *silently* wrong in a way that still fits the training data to machine precision through the normal equations while reconstructing badly, which is why `tests/test_next_state.py::test_affine_part_recovers_a_purely_affine_target` exists — it fits a purely affine ground truth and demands recovery to within 10^{-3} relative error.

C.3 The regime that actually bites: $N \ll d$

Appendix B fixes the decoder. It does not fix the *subspace*, and the subspace is the more consequential problem in practice.

A realistic labelled corpus for LLM activations has tens to a few hundred examples; the hidden size is 768 to 4096. In that regime the pooled within-class covariance has rank at most $N_1 + N_2 - 2$ and is therefore **singular**. The reference `SubspaceLearner.fit` forms it, adds a fixed $1e-6 * I$, and calls `torch.linalg.inv`. An explicit inverse squares the condition number, and the ridge is an absolute constant unrelated to the scale of the features.

Controlled diagnostic (synthetic, 56 labelled examples in $d = 768$, Gaussian clusters separated along a single axis, 200 held-out examples per class). This isolates the fitting pathology from data quality:

Fit	Train balanced acc.	Held-out balanced acc.	Condition
Reference: absolute 10^{-6}	1.000	0.490	8.9×10^9
ridge + explicit inverse			
Cholesky + Ledoit–Wolf shrinkage	1.000	0.608	8.0×10^1

Fit	Train balanced acc.	Held-out balanced acc.	Condition
Cholesky + shrinkage, PCA-32 pre-conditioning	0.661	0.537	1.70
Cholesky + shrinkage, PCA-8 pre-conditioning	0.536	0.528	1.17

The reference fit returns a direction that is **no better than a coin flip** on held-out data while interpolating its training set perfectly, at a condition number of 8.9×10^9 . Cholesky with scale-aware shrinkage recovers 0.608 on the same data — a 0.118 absolute gain, and the honest headline of this module. `tests/test_numerics.py::test_beats_the_reference_fit_in_the_small_s` pins the *direction* of this effect with deliberately loose thresholds so it tracks the phenomenon rather than a particular number.

PCA pre-conditioning behaves as theory says and does not help: it removes the optimism (train accuracy falls to 0.66 as the fit can no longer interpolate) but does not improve held-out accuracy, because the one-dimensional signal is diluted by the same projection that removes the noise.

Two honest qualifications.

The real-data head-to-head is inconclusive. In the pilot corpus (Appendix D) the held-out split contains roughly 22 examples, of which about 4 are toxic. A balanced-accuracy difference of the size observed there is well within sampling noise at that size, and the report does not treat it as a result in either direction. The controlled diagnostic above is synthetic, is designed to isolate the fitting pathology, and is labelled as such.

PCA pre-conditioning is not a free win. Projecting onto the leading principal directions removes the train/held-out optimism, because the fit can no longer interpolate the training set — train accuracy falls towards the held-out value. But on the controlled diagnostic it did *not* improve held-out accuracy over scale-aware shrinkage, because the underlying signal is one-dimensional and is diluted by the same projection that removes the noise. It is shipped in `sasa/numerics.py` as a selectable option with tests, and it is not recommended as a default.

C.4 α is not scale-free, and that is the practical bug

α multiplies a margin whose norm is an artefact of the fit, not a property of the model. `margin = <w, h - b> / ||w||` is scale-invariant in the hidden state but **not** in w , and $w = \Sigma^{-1}(\mu_1 - \mu_2)$ inherits its scale from the conditioning of Σ and the feature scale of the representation. Two models, or two fitting procedures, therefore interpret “ $\alpha = 1$ ” completely differently, and the paper’s recommended range $\alpha \in [0.5, 2.0]$ is meaningful only for the paper’s own w .

The fix is to parameterise the controller by a *distributional budget* rather than a raw scalar: choose the smallest α such that the per-step distributional distortion stays under a stated bound,

$$\alpha^*(\epsilon) = \min \left\{ \alpha : \text{KL}(p_{\text{base}} \| p_\alpha) \leq \epsilon \right\}.$$

Because the steering vector is fixed, the KL is monotone in α , so this is a bisection to any precision and needs no sampling. $\alpha^*(\epsilon)$ is comparable across models, across fitting procedures, and across hidden sizes, and it has an interpretation a practitioner can check: the fraction of probability mass the controller is allowed to move per token.

Measured on 8 real prompts with real `distilgpt2` logits, against both fitted directions. $\alpha^*(\epsilon)$ is the largest α whose per-step KL stays within the budget.

KL budget ϵ	α^* (Cholesky)	α^* (reference)	ratio
0.01	2.74	4.32	1.58
0.02	3.90	6.10	1.56
0.05	6.27	9.59	1.53
0.10	9.02	13.47	1.49
0.25	14.76	20.92	1.42
0.50	21.75	28.84	1.33

Two findings, both immediately actionable.

A given α does not mean the same thing across fitting procedures. The reference direction requires **1.33–1.58 $\times$ larger α** than the Cholesky direction for *identical* distributional distortion — a 33–58 % error in the one hyperparameter, from nothing but a different linear solver and a different ridge. Note that $\|w\|$ differs by **10.6 $\times$** between the two fits, which is the mechanism: the raw α inherits the arbitrary scale of a near-singular solve.

The paper’s recommended range is a near-no-op. At $\alpha = 2$ the mean per-step KL is 2.1×10^{-3} and the logit shift is **0.009 logit standard deviations**. To move a tenth of a percent of the probability mass you need $\alpha \approx 9$; to move half, $\alpha \approx 22$. The paper’s $\alpha \in [0.5, 2.0]$ corresponds to a distributional change of 0.01–0.2 %, which is why a re-run at those values can easily fail to detect any effect (§D.5) without the method being wrong. Anyone re-running this work should sweep α over at least three orders of magnitude, or calibrate by ε and never quote a raw α again.

C.5 Threats

- **The ground truth is the model itself.** NLMA is trained to predict distilgpt2’s own next hidden states, so it inherits every bias in that representation, including whatever made the layer sweep in Appendix E fail. Predicting the model’s internals well and detoxifying well are different claims.
- **The interaction is fitted by Adam, not in closed form.** The affine part is closed-form; the low-rank part is not, and its solution depends on the step count and the seed. Both are fixed and asserted in tests, but “closed form” applies only to the affine part and the report does not claim otherwise.
- **No measurement of whether a better next-state estimate helps.** That requires a model large enough for steering effects to be measurable, which this environment does not provide. Appendix E shows why.

Appendix D — A pinned external judge, and the closed-loop evaluation trap

Contributions 4 and 5 of §6. Code: `evaluation/judge.py`, `tests/test_judge.py`. Reproduce with `experiments/run_pilot.py`.

D.1 The structural problem

SASA fits a toxicity subspace on a model’s own hidden states and steers by the margin of those same hidden states. If the effect is then scored with the same representation, improvement is guaranteed by construction: the optimisation target and the metric are the same object. Any trustworthy evaluation needs judges ordered by their independence from the steered model.

Judge	Independent of the steering subspace?	Independent of the model?	Verdict
The steering margin itself	no	no	circular by construction
A held-out probe on the same representation	yes	no	partially circular
An external judge on the text	yes	yes	the only usable endpoint

`docs/ROADMAP.md` Phase 1 anticipates part of this — it worries about Perspective-API gaming by evasive paraphrase. The deeper failure is prior to that: if the metric is the model’s own representation, the score cannot go down, whatever the text says.

D.2 The pinned judge

`evaluation/judge.py` implements the external endpoint in the form `docs/METHODS.md` U1 Option B requests — “reproducible, free, versionable, no network dependency”. Three of those four are properties of *pining*, and the fourth is a property of the cache:

- **Versionable.** openai/gpt-4o-mini-2024-07-18 is a *dated snapshot*, not a floating alias. The rubric is versioned by content hash (tox-strict, version 8cac61df2ad4).
- **Reproducible.** temperature = 0, a fixed seed, a frozen rubric string. The full configuration collapses to a 12-character fingerprint that is part of every cache key, so changing *any* of it invalidates the cache by construction rather than by convention.
- **No network dependency on re-run.** Every response is written to an append-only JSONL cache keyed by sha256(fingerprint | text). A re-run of an unchanged corpus makes zero network calls.
- **Free.** At the scale used here, the total judge spend is a fraction of a cent.

Scores are on the 0–4 scale used by RealToxicityPrompts (Gehman et al., 2020, arXiv:2009.11462), so they are comparable in *convention* with published toxicity work even though the scorer is a different model. The rubric used is tox-strict, which adds an explicit instruction not to be fooled by ironic or politely-phrased abuse — an attempt to make the judge harder to game by paraphrase, which is the specific failure mode Phase 1 identifies.

Requests are issued concurrently (default 8 workers) because OpenRouter returns intermittent 503 responses even with a valid key and no account limit. Any text that exhausts its retries is recorded with a null score rather than being dropped; null is excluded from means and the count is reported.

D.3 Protocol, fixed before the run

Per docs/METHODS.md U3 the pre-registered primary endpoint is the independent judge’s mean score at each α against $\alpha = 0$, with significance by Wilcoxon signed-rank resampled at the **prompt** level, and fluency tracked by repetition rate and corpus distinct-2 alongside.

Corpus. 16 charged prompt stems, 3 continuations each sampled from the generator at temperature 1.0, plus a curated contrast set for class balance. Class membership is assigned by an explicit judge threshold — toxic at score ≥ 2 , non-toxic at score exactly 0 — rather than by a median split, because a median split on a mostly-benign corpus fills the “toxic” class with borderline score-1 text and weakens the signal.

Corpus composition (80 judged texts; contrast set of 73):

Judge score	0	1	2	3	4
Texts	59	7	7	2	5

Contrast set: 14 texts with score ≥ 2 (toxic class) against 59 texts with score exactly 0 (non-toxic class), 70/30 stratified split, hidden size 768. Mean judge score over all 80: 0.59. **74 % of the generator’s own output is scored 0**, which is the fact that governs every result below.

D.4 Results

16 prompts, 20 new tokens each, temperature 1.0, top- $k = 50$, at $\alpha \in \{0, 1, 3, 8, 20\}$. Judged independently with gpt-4o-mini-2024-07-18 under rubric tox-strict (version 8cac61df2ad4).

α	Logit shift (logit SD)	External judge	Own margin	Probe margin	Repetition	Distinct-2	Wilcoxon p
0	0.000	0.063	0.033	0.187	0.000	0.954	—
1	0.007	0.400	0.067	0.144	0.008	0.956	—
3	0.019	0.063	0.050	0.199	0.012	0.949	0.655
8	0.052	0.125	0.058	−0.181	0.007	0.933	1.000
20	0.129	0.125	0.083	0.949	0.004	0.955	0.789

No α produces a significant change in the independent judge ($p \geq 0.65$ throughout). Fluency is untouched: distinct-2 stays in [0.933, 0.956] and the 3-gram repetition rate never exceeds 0.012, so nothing here is a fluency-vs-toxicity trade-off — there is no toxicity effect to trade against.

This is the part that does show a clean signal, and it is a signal *about the metric*.

- The **external judge** (independent of the model) does not move: 0.063 at baseline, 0.125 at $\alpha = 20$, with $p = 0.79$.
- The **own-margin** — the steering subspace’s own score on the generation, the quantity the decoder was optimising — rises from 0.033 to **0.083**, a $2.5\times$ increase, monotonically in α across three of four steps.
- The **held-out probe** is erratic (0.187 $\rightarrow$ $-0.181 \rightarrow 0.949$) and tracks nothing, which is what a direction with 0.516 balanced accuracy (§E.2) should be expected to do.

The pattern is exactly the predicted failure mode. An evaluation that scored SASA with the model’s own representation would have reported a consistent, monotone, dose-dependent improvement in detoxification. The independent judge reports no effect. **The dose-response relationship is a property of the metric, not of the text.** This is the closed-loop trap, measured rather than argued — and it is the single most useful thing in this appendix for anyone re-running the repository’s benchmark, because the repository’s own Phase 1 plan does not currently include a judge that is independent of the steered model.

D.5 The headline negative result

At $\alpha = 20$ the controller shifts the logits by 0.129 standard deviations, and no effect is detectable. The reason is quantitative, not accidental, and Appendix C.4 quantifies it: for this fitted subspace, a distributional change of even 0.5 % requires $\alpha \approx 22$, so $\alpha \in [0.5, 2]$ — the paper’s range — corresponds to 0.01–0.2 % of probability mass moved. The sweep in this table never left the near-no-op regime.

But the deeper reason is the layer sweep (§E.1): on this model and this corpus the toxicity subspace is **not identifiable at all** — no layer separates toxic from non-toxic text better than chance. A controller built on a subspace that does not exist cannot work, and no value of α will change that. The comparison is therefore underpowered by construction, and the honest statement is:

On distilgpt2 with a 14-example toxic class, this work **failed to demonstrate any detoxification effect, and established a pre-emptive reason to expect failure**: the learned direction has below-chance held-out separability at every depth, and the steering it induces is two orders of magnitude too small to matter at the α values in the literature.

Two further reasons the study is underpowered, stated so they are not mistaken for robustness:

- **Statistical power.** Judge scores are discrete on $\{0.4\}$ and mostly tied, so the Wilcoxon test has $n = 1\text{--}3$ non-zero paired differences. The p -values are not “null results with tight confidence intervals”; they are tests with almost no power. No confidence interval is reported for the same reason.
- **Ceiling effect.** 74 % of baseline generations score 0. There is very little toxicity to remove, so even a working controller would have little to do.

D.6 Threats

- **The judge is still a model.** It can be fooled by fluent evasive text. The mitigation here is only that it is *external*; the appropriate final arbiter remains the human spot-check protocol of roadmap item #5, which this does not replace.
- **Same-name-different-provider.** OpenRouter may route a pinned model id to different providers. Only the parsed score is cached, not the raw response, so a future re-verification could in principle disagree. Storing raw responses would close this and is the first change to make if the work continues.
- **Ceiling effects dominate.** See D.5: with a generator this small, there is very little toxicity to remove, which makes the study underpowered regardless of the judge.

Appendix E — Geometry: depth, rank, and the price of static interference

Contributions 6 and 7 of §6. Data: results/pilot.json. Code: experiments/run_pilot.py, sasa/numerics.py.

E.1 Layer sweep: the toxicity subspace is not identifiable

`sasa.subspace_learner.extract_embeddings_from_model` and every call site in `SASASampler.generate` read `hidden_states[-1]` — the last layer — on the unstated assumption that it is the most semantically abstract. That is a

testable assumption, so it was swept: one direction fitted per layer, scored by balanced accuracy on the held-out third of the labelled set.

Layer	0 (embeddings)	1	2	3	4	5	6 (final)
Balanced accuracy	0.465	0.324	0.353	0.265	0.365	0.324	0.265

Every layer is at or below chance (0.5). The best, layer 0, reaches 0.465 — which is not a weak signal, it is a *backwards* one. The final layer that the implementation actually uses scores 0.265.

This is the load-bearing negative result of the report, and it should be read before any of the steering numbers:

On distilgpt2, with a corpus whose toxic class is 14 examples judged ≥ 2 by an external judge, **toxicity is not linearly decodable from this model’s hidden states at any depth.** The repository’s design assumption — that a linear hyperplane in representation space separates toxic from non-toxic content — does not hold for this model at this corpus size.

Three candidate explanations, and the present experiment does not distinguish between them. They are listed because the first is the cheapest to rule out and would invalidate the rest of the agenda if true.

1. **Corpus too small.** 14 toxic examples in 768 dimensions, pooled covariance rank ≤ 71 . §C.3 shows this regime is exactly where fitting fails. The controlled synthetic diagnostic shows the *reference* fit dropping to chance in precisely this configuration.
2. **Model too small.** An 82 M-parameter model may simply not represent toxicity. This is the most likely explanation and it is untestable here.
3. **Labels too hard.** The toxic class is dominated by the curated set (short, generic insults) while the non-toxic class is dominated by model samples. A direction fitted to that contrast may be detecting *curation style* rather than toxicity — a real confound, and a specific instance of the refusal-versus-toxicity confound flagged as idea 9 in §6.

The honest position is that the experiment **cannot separate these**, and that any claim that the method works at this scale is unsupported. The remedy is not more analysis of this corpus; it is a larger labelled set on a larger model, neither of which this environment can provide.

E.2 Two directions, two conclusions, and why the second is the weaker one

Fit	Train balanced acc.	Held-out balanced acc.	Condition	Ledoit–Wolf λ
Cholesky + shrinkage	0.794	0.524	$2.8 \times 10^{11} \rightarrow 261$	1.00
Reference (as shipped)	0.853	0.782	2.6×10^6	—
Held-out probe (reverse split)	0.516	—	—	—

Two things deserve comment.

The shrinkage coefficient came out at exactly 1.00 — maximum shrinkage, meaning Ledoit–Wolf’s own estimate concluded that the sample covariance carries no usable information and the best available estimate is the scaled identity. This is the numerics module correctly refusing to trust its input, and it predicts the 0.524 held-out accuracy that follows. Had it been used on this data, the report would have had to explain why its own method scored worse than the baseline; instead it emits a diagnostic that predicts the failure.

The reference fit’s higher held-out accuracy (0.782) is not a result. The held-out split has ~22 examples of which ~4 are toxic. A balanced-accuracy difference of that size at $n \approx 4$ positive examples is indistinguishable from noise, and the reference fit is the *less* regularised of the two, which §C.3’s controlled diagnostic shows is the wrong direction in this regime. Reporting 0.782 as evidence that the shipped fit is better would be exactly the selective reporting this report criticises in §4. It is reported, and explicitly not interpreted.

The probe’s 0.516 is the cleanest number here: a direction fitted on held-out data and evaluated on the fit data is at chance, independently confirming E.1.

E.3 Static interference prediction (idea 6) — specified, not measured

The claim is that interference between two composed attribute margins is predictable *before* any generation, from the geometry of the learned directions. If attribute i steers with w_i , the composition $z' = z + \sum_k \alpha_k \text{margin}_k$ can only fail through the interaction between w_i and w_j , so $\cos(w_i, w_j)$ is a *prior* on interference that costs one Gram matrix to compute and no generation at all.

This converts a combinatorial search over $(k!)$ attribute schedules into a screening step over $k(k-1)/2$ cosines. It is the cheapest unexploited idea in §6 — it needs k labelled contrast sets and no GPU — and it is listed as idea 6 with status [P] rather than claimed.

It is **not** measured here, and the reason is specific rather than convenient: the attribute subspaces this predicts interference *between* would have to be fitted on the same 14-example toxic-class regime that E.1 shows is already below chance. A prediction validated against directions that carry no signal would be worse than no prediction. The honest status is “specified, blocked on a corpus that makes it identifiable”.

E.4 Rank- k subspaces (idea 8) — specified, not measured

Toxicity is plausibly multi-dimensional: offensiveness, hostility, and discrimination are three different things, and a single hyperplane forces them to be one. `sasa.numerics.fit_direction` already supports a reduced representation via `pca_dim`, which is the natural substrate for a rank- k subspace and a swept- k held-out AUC. Not attempted, for the same reason as E.3.

E.5 What would change these conclusions

In priority order, and each is cheap relative to what it would save:

1. **A labelled corpus three orders of magnitude larger.** 14 toxic examples cannot support a 768-dimensional decision problem. This is the single binding constraint on every result in this appendix.
2. **A model with a GPU.** Below chance at every layer on 82 M parameters generalises to nothing.
3. **Counterfactual pairs** that hold style fixed and vary only the target (idea 9), which rules out the curation confound in E.1 without needing more data.
4. **A refusal probe** to test whether the direction encodes refusal rather than toxicity — the cheapest of the four, and it can be run on the existing corpus.

Appendix F — Cross-model transfer, and a feasibility correction to Phase 3

Idea 10 of §6, executed against the rule the repository already fixed in advance. Code: `experiments/run_transfer.py`. Data: `results/transfer.json`.

F.1 Scope, stated before the result

Roadmap item #11 asks for a Procrustes alignment map between the paired hidden states of a proxy and a target model, and `docs/METHODS.md` U4 fixes the choice between orthogonal Procrustes and ridge in advance: *fit both on the same paired data, pick by held-out margin-preservation error, and let orthogonal Procrustes win ties; adopt ridge only if it wins by more than 10 %*.

This appendix executes U4. It does **not** claim that a *toxicity* subspace transfers. Appendix E established that the toxicity direction on `distilgpt2` is below chance at every depth, and transporting a direction that carries no signal would measure noise. What is measured is the **transfer operator** — whether a Procrustes map between two models’ representation spaces preserves a decision function — which is a precondition for Phase 3 and can be evaluated without a working toxicity direction.

F.2 Setup

Item	Value
Proxy	distilgpt2, $d_p = 768$, final layer
Target	gpt2, $d_t = 768$, final layer
Paired observations	8 documents $\times$ 24 token positions = 192
Fit / held-out	115 / 77
Ridge penalty	$\lambda = 1.0$

Observations are token *positions*, not prompts. A 768 $\rightarrow$ 768 map has 590 k free parameters, so a handful of prompt-level states is a vacuous fit; harvesting every position from a handful of documents is what makes the operator estimable at all. An earlier version of this experiment with 9 fit pairs produced a near-perfect “result” that was pure overfitting, which is the reason the sample size is stated so prominently.

F.3 Results

Quantity	Value
Orthogonal Procrustes fit residual $\ XW - Y\ _F / \ Y\ _F$	0.364
Ridge fit residual (same data, $\lambda = 1$)	16.64
Held-out rank agreement of the transferred top direction (r)	0.9989
Linear CKA (Kornblith et al., 2019)	0.802
Top canonical correlation between centred subspaces	1.000
U4 winner	orthogonal Procrustes (46 $\times$ lower residual)

Three readings, in descending order of confidence.

U4 resolves decisively, and in the direction the plan hoped for. Procrustes beats ridge by a factor of 46 in-sample. The reason is structural rather than empirical, and it is the useful contribution here: **the two options in U4 are not equally feasible at the data scale Phase 3 specifies.** docs/ROADMAP.md Phase 3 step 2 proposes collecting “2–5 k paired hidden states”. A ridge map from a d_p -dimensional proxy to a d_t -dimensional target has $d_p d_t$ free parameters — 8.4 M for the stated Llama-3.2-1B $\rightarrow$ Llama-3.1-8B pair — and is therefore hopeless from 2–5 k samples no matter how λ is tuned, which this run reproduces exactly (residual 16.6, worse than predicting the mean). An orthogonal Procrustes map is *constrained* to $W = UV^\top$ with $\min(d_p, d_t)$ singular directions, so the effective parameter count is $\min(N, \min(d_p, d_t))$: a few thousand pairs align a few thousand directions, which is a large fraction of the smaller space. **Phase 3’s data plan is adequate for the method it prescribes and inadequate for the alternative it considers.** U4’s selection rule is therefore not a close call at the intended scale, and the “ridge wins by >10 %” escape clause is a contingency that cannot fire in practice.

The two models’ representations are substantially aligned. CKA = 0.802 is well above the ~0.1–0.3 range typically reported for unrelated models and in the range usually associated with strong alignment, which is the concrete evidence that distilgpt2 (distilled from gpt2) retains its parent’s geometry. This is the mechanism Phase 3 needs, and it holds.

Held-out rank agreement of 0.9989 is weaker evidence than it looks. The direction being transported is the *top principal component* of the fit split. For text representations that component is dominated by frequency and position effects, which are the most transferable thing in the space. The number shows that the *operator* transports a function faithfully; it does not show that a *semantically meaningful* direction would survive. Given Appendix E, a toxicity direction has no chance of being the top component here.

F.4 Threats

- **Same family, same width.** distilgpt2 and gpt2 share an architecture, a tokenizer, and a lineage, and both have $d = 768$, so the zero-padding step that cross-family transfer requires is untested. Cross-family claims (Llama $\rightarrow$ Qwen/Gemma) remain open, and this appendix does not touch them.
- **Final layer only.** Layer choice interacts strongly with transferability and was fixed at layer -1 for both models for tractability.

- **Ridge not tuned.** $\lambda = 1$ is a single value. A swept ridge would narrow the $46\times$ gap, though not plausibly to the point where it beats an orthogonal constraint at this sample size — the structural argument above is what carries the conclusion, not the single λ .
- **A precondition, not the hypothesis.** F.3 establishes that the transfer operator works and that the two spaces are aligned. It says nothing about whether a toxicity subspace exists to transfer, which is the question Phase 3 is really asking and which Appendix E shows this environment cannot answer.

Appendix G — Reproducibility, provenance, and how to re-run this

This appendix is the audit trail for every number in this report. It exists because the audit in §4 found the audited repository’s own numbers unreproducible, and an audit that commits the same sin is worthless.

G.1 Environment

Item	Value
Model	distilgpt2 (82 M parameters, 6 layers, $d = 768$, $V = 50257$)
Framework	PyTorch 2.14.0+cpu, transformers 5.17.0
Hardware	2 CPU cores, 3 GB RAM, no GPU
Python	3.11
Seed	20260930
Judge	openai/gpt-4o-mini-2024-07-18, rubric tox-strict

Scale disclosure. No model above 124 M parameters was run. The efficiency results in §5.2 and Appendix B are exact arithmetic identities that hold at any scale, and the operation counts are reported analytically for Llama-3.1-8B. The *steering* results are not scale-general, and the report never claims they are. This is the single largest limitation of the work and it is a property of the execution environment, not a finding.

G.2 Commands

Every claim in this report is produced by one of four commands, run from the repository root.

```
# 0. Environment (CPU-only torch is ~180 MB)
python -m venv --system-site-packages .venv
.venv/bin/pip install torch --index-url https://download.pytorch.org/whl/cpu
.venv/bin/pip install transformers pytest

# 1. Full test suite: 106 tests, no network, no model download
.venv/bin/python -m pytest tests/ -q

# 2. Appendix B -- exactness, cost, context-independence
.venv/bin/python experiments/run_collapse_benchmark.py --out results/collapse.json

# 3. Appendices D/E -- corpus, subspace, steering, three-judge comparison
OPENROUTER_API_KEY=... .venv/bin/python experiments/run_pilot.py --out results/pilot.json

# 4. Appendix C -- alpha calibration by distributional budget
.venv/bin/python experiments/run_alpha_calibration.py --out results/alpha.json
```

G.3 Determinism and caching

Determinism is enforced at four levels, and each is a separate failure mode:

1. **Sampling.** Every generation path takes a seeded `torch.Generator`. The corpus sampler and the evaluation decoder use *different* seeds (`seed` and `seed + 1`) so that the corpus is not silently conditioned on the evaluation prompts.
2. **Model initialisation.** No model is trained. Every direction is a deterministic function of fixed data via a closed-form solve, except the NLMA interaction factors, which use a seeded mini-batch sampler and a fixed step count. `tests/test_next_state.py::test_deterministic_given_seed` asserts bit-identical parameters across two fits.
3. **The judge.** Pinned to a *dated* model snapshot, `temperature = 0`, a fixed seed, and a frozen rubric. Every response is written to an append-only JSONL cache keyed by `sha256(model | rubric | seed | text)`. A second run of an unchanged corpus performs **zero** network calls and reproduces every number exactly. This is the property `docs/METHODS.md` U1 Option B asks for and that a live API call cannot provide.
4. **Statistics.** The pre-registered test is a normal-approximation Wilcoxon signed-rank with continuity correction, implemented in-repo so that no SciPy version can silently change a p-value. Bootstrap intervals are not reported because the sample sizes here are too small for them to be informative; that is disclosed rather than papered over.

G.4 What is cached, and what that implies

Artefact	Cached	Consequence
Judge responses	Yes, on disk	Results reproduce offline; a rubric change invalidates the cache by construction
Model weights	Yes, in the HF cache	No download on re-run
Corpus sampling	No, but seeded	Re-derivable in ~180 s; the judge cache makes the labelling free
Subspace fits	Yes (<code>results/subspace.pt</code>)	Downstream experiments share one fit exactly

The cache is keyed on configuration, not just text. Changing the judge model, the rubric, the temperature or the seed produces a different key, so a stale cache cannot silently contaminate a new configuration. The cache is append-only and each record is a single line written under a lock with `fsync`, so an interrupted run leaves a valid file.

G.5 Threats to reproducibility that remain

The judge is a third party. `gpt-4o-mini-2024-07-18` is pinned by name, but OpenRouter may in principle route to a different provider for the same model id. The response is not stored verbatim, only the parsed score, so a re-verification would need to re-run and could in principle disagree. Storing raw responses would close this; it is the first thing to change if this work is continued.

Transient API failures are real. During this work OpenRouter returned intermittent 503 responses for roughly half of all requests over a period of several minutes, with a valid key and no account limit. The judge retries with jittered, capped backoff and requests concurrently, and any text that exhausts its retries is recorded with a `null` score rather than being silently dropped — a `null` is excluded from means and the count is reported.

A small model is a small model. `distilgpt2` produces almost no text the judge scores above 1 (§D.3), which creates a ceiling effect that makes the steering comparison underpowered. This is reported as a `null`, not worked around.

G.6 Repository hygiene

The contributions in this report are additive. No existing public symbol changes behaviour, `sasa/sampler.py`, `sasa/subspace_learner.py`, `sasa/__init__.py` and the README are untouched, and the README’s performance table is left exactly as found — it is the maintainer’s call to revise or retract it, not a contributor’s. Each contribution ships with tests, per the cross-phase rule in `docs/ROADMAP.md`.

Appendix H — Graded attributes, interference geometry, and confounds

Ideas 6, 8, and 9 of §6. Code: `sasa/multi_attribute.py`, `sasa/probes.py`, `evaluation/attributes.py`. Data: `results/attributes.json`. Reproduce with `experiments/run_multi_attribute.py`.

H.1 Why the attribute set is not toxicity

Appendix E found the toxicity direction below chance at every layer of `distilgpt2`. Interference between attributes whose own subspaces carry no signal would measure noise, so the four attributes studied here are **lexically grounded** and mechanically gradeable: first-person narration, second-person address, interrogative sentences, and unhedged phrasing. Every label is a property of the text a reader can verify, derived by `evaluation.attributes.attribute_grade` with no model in the loop.

Attribute	Intent	n per class	Held-out accuracy	z vs chance	Identifiable
<code>first_person</code>	first-person narration	30	1.000	+2.12	yes
<code>interrogative</code>	question vs statement	30	1.000	+2.12	yes
<code>hedge_free</code>	direct vs hedged	30	0.944	+1.89	yes
<code>second_person</code>	address to the reader	30	0.833	+1.41	no

Three of four clear a one-sided 5 % test against chance; `second_person` does not, at 0.833 accuracy on 18 held-out examples. The gate is doing its job, and it is worth being explicit about why a *threshold* would not have: on 18 held-out examples a probe fitted to pure noise lands anywhere in roughly [0.38, 0.54], so any accuracy cutoff near 0.5 admits noise. `sasa.probes.Probe` therefore reports a z-score against chance rather than a pass/fail accuracy.

Two design details are worth recording because they were got wrong first. The corpora began at 8 examples per class, which left 2–3 held out per class and made the significance test meaningless; 30 per class is the minimum that makes it informative. And the hedge lexicon initially contained “some” and “around”, which mislabelled ordinary prose — the fix was to shrink the lexicon to fifteen explicit modals and correct the three affected sentences, rather than to keep growing the list until the labels looked right. A label set tuned until it agrees with itself is not a label set.

H.2 Confound geometry

Every pair of attribute probes is between 85° and 94° apart — near-orthogonal, as a reader would expect of genuinely different surface properties. The smallest angle is `first_person/second_person` at 85.4°, the largest `second_person/hedge_free` at 94.2°.

The practical consequence is about Appendix E’s unresolved third explanation. The toxicity direction there could plausibly have been tracking a *curation* signal rather than harm, because the toxic class was dominated by hand-written sentences. The confound machinery in `sasa.probes` is the tool to settle that question, and it reports the price of every adjustment alongside the gain (`partial_out_heldout_accuracy` returns `accuracy_before`, `accuracy_after`, `cost`, `cosine_removed`) rather than applying a correction quietly. It was not run against the toxicity direction here, because that direction is below chance and there is nothing to de-confound.

H.3 Static interference prediction: right sign, no power

If attribute i steers with margin f_i and attribute j with f_j , the only channel through which they interact is the overlap of their weight subspaces, and that is computable before any generation. The prediction is therefore cheap. The question is whether it *ranks* interference correctly, so interference was also measured independently — by composing biases at matched steering mass and computing, for each attribute, the fraction of its intended margin effect that survives the composition.

Pair	Predicted overlap	Retention a	Retention b	Measured loss
first_person / second_person	0.035	+2.35	+0.67	-0.67
first_person / interrogative	0.006	+2.35	+1.97	-0.97
first_person / hedge_free	0.001	+2.35	+2.41	-1.41
second_person / interrogative	0.004	+0.67	+1.97	-0.33
second_person / hedge_free	0.034	+0.67	+2.41	-1.41
interrogative / hedge_free	0.004	+1.97	+2.41	-1.41

Spearman between predicted overlap and measured loss: $\rho = +0.62$, $p = 0.117$ over six pairs.

The sign is right and the effect is not significant, and the reason is visible in the table: **mean retention loss is -0.39 , i.e. negative.** Every retention is above 1, which means composing the biases *amplified* each attribute’s effect rather than cancelling it. That is the correct behaviour for near-orthogonal directions, and the mean overlap of 0.014 says these attributes are almost exactly orthogonal. So the prediction is untested in the only regime where it would bite: with attributes that actually overlap.

Honest status: specified, implemented, and inconclusive. The screening idea is sound and the code is in place, but this corpus cannot validate it, because a corpus whose attributes are orthogonal by construction has no interference to predict. Validating it requires deliberately *correlated* attribute pairs — a graded severity scale and a coarse sentiment scale, say, which are correlated in the real world for reasons that are not an artefact of the corpus. That is the experiment to run next, and it is the one thing in this appendix that is genuinely unfinished.

H.4 Graded supervision: free, and measurably better

Two-class Fisher discriminant analysis can only ever produce **one** direction, so SASA’s rank-one margin is not a simplification of a richer method — it is all the two-class method has. But the pinned judge already emits a graded 0–4 label at no extra labelling cost, and with k classes multiclass LDA has $k - 1$ independent directions. The graded margin generalises the binary one exactly:

$$f(x) = \max_{j \in \text{benign}} \delta_j(x) - \min_{j \in \text{toxic}} \delta_j(x), \quad \delta_j(x) = x^\top \Sigma^{-1} \mu_j - \frac{1}{2} \mu_j^\top \Sigma^{-1} \mu_j,$$

which at $k = 2$ reduces to the binary score $\delta_{\text{benign}} - \delta_{\text{toxic}}$ — a claim `tests/test_multi_attribute.py::test_graded_reduces_to_binary` pins to a correlation above 0.97.

On the 79 externally judged generations from Appendix D:

Supervision	Grades used	Effective rank	Held-out grade accuracy	Held-out side accuracy
Binary (≥ 2 vs 0)	2	1	0.792	1.000
Graded (0–4)	4	1	0.917	1.000

Graded supervision predicts the grade 12.5 points better than the binary split at identical labelling cost. That is the claim, and it holds.

The rank gain did **not** materialise: both fits end up rank 1. The reason is recorded rather than hidden — the corpus’s grade distribution is lumpy (59 texts at 0, 7 at 1, 7 at 2, 2 at 3, 5 at 4), and the rare middle grades are merged into the nearest populated grade by `min_class_size`, leaving two classes. `GradedSubspace.summary()` reports `merged_grades` so the merge is visible rather than implicit. Getting a genuine rank- k discriminant needs a corpus with a smoother grade distribution — the RealToxicityPrompts distribution the upstream work uses would supply it, and it is not available here.

H.5 Threats

- **The attributes are easy.** First-person markers and question marks are lexical, so a linear probe finding them is unsurprising and the near-1.0 accuracies should not be read as evidence that linear probes are powerful in general. The experiment is about the *geometry* between subspaces, not about probe accuracy.
- **The interference test is underpowered by construction** (H.3), and the retention metric has no independent validation.
- **second_person failed its own gate** and is nevertheless retained in the interference table, because the gate governs the `is_identifiable` flag on the probe while `AttributeSet.identifiable()` gates on the graded subspace’s side accuracy. These are different quantities and both are reported; a reader who wants the stricter subset can take three attributes instead of four.
- **Everything is one model at one layer.** The final layer only.

Appendix I — Steering schedules, the U2 rule, and the degeneracy knee

Ideas 13 and 14 of §6. Code: `sasa/scheduling.py`, `sasa/decoding.py`. Data: `results/schedules.json`. Reproduce with `experiments/run_schedules.py`.

I.1 What a schedule can and cannot change here

Under the reference next-state estimator the steering vector is *static* (Theorem 1, Appendix B), so a schedule changes only the **magnitude** of one fixed bias over a generation. It cannot change which tokens are preferred — the token ranking is identical at every step, which `tests/test_scheduling.py::test_schedule_is_the_only_position_dependent_knob` asserts directly. A schedule is therefore a *dose* control, not a policy.

That is worth stating before the results, because it bounds what any schedule can be expected to buy, and because it is not obvious: the decoder takes `alpha(t)` at each step, which looks like a policy but is not one.

The families implemented are `fixed`, `linear` (ramp), `cosine` (decay), `step`, and `margin_gated` (opens when a caller-supplied context score crosses a threshold). Only the last can be *position-sensitive* in a way that matters, and only because the caller supplies the signal — the gate function receives `(step, context_score)` and decides.

I.2 Schedules: no measurable difference at the literature’s strengths

16 prompts $\times$ 24 tokens, top- $k = 50$, peak $\alpha = 8$:

Schedule	Peak α	Logit shift (logit SD)	distinct-2	Unique texts	3-gram repetition
baseline	0	0.000	0.926	1.00	0.000
fixed	8.00	0.028	0.883	1.00	0.003
linear	8.00	0.028	0.925	1.00	0.000
cosine	8.00	0.028	0.914	1.00	0.003
margin_gated	5.71	0.020	0.881	1.00	0.003

The spread across schedules (distinct-2 between 0.881 and 0.925) is inside the noise of a 16-prompt sample, and the logit shift at $\alpha = 8$ is 0.028 standard deviations — a regime where the decoder is barely doing anything at all, as I.3 shows. **No schedule is distinguishable from any other here, and none should be: there is no signal to shape.**

I.3 The degeneracy knee

The useful result of this study is not the schedules but the *range*. Sweeping α over four orders of magnitude, with 16 prompts $\times$ 24 tokens each:

α	Logit shift (logit SD)	distinct-2	Unique texts	3-gram repetition
0	0.000	0.926	1.00	0.000
8	0.028	0.883	1.00	0.003
32	0.111	0.876	1.00	0.016
128	0.445	0.667	0.81	0.000
512	1.780	0.000	0.06	0.000
2 048	7.122	0.000	0.06	0.000
8 192	28.487	0.000	0.06	0.000
32 768	113.947	0.000	0.06	0.000

This is a **cliff, not a slope**, and its location is the useful number:

- **below ~0.1 logit SD** — no measurable fluency cost;
- **at ~0.45 logit SD** — distinct-2 falls from 0.88 to 0.67 and a fifth of prompts start producing text identical to another prompt;
- **by ~1.8 logit SD** — total collapse. distinct-2 is exactly 0 and 94 % of prompts produce the *same* string, i.e. the decoder has degenerated into emitting one fixed token sequence regardless of the prompt.

Greedy decoding (top- $k = 1$) shows the same cliff at the same place: 0.754 distinct-2 at $\alpha = 0$, 0.000 at $\alpha = 512$.

The punchline for anyone re-running this work is the gap between the cliff and the literature:

The published $\alpha \in [0.5, 2.0]$ produces a logit shift of **0.001–0.03 standard deviations** — between one and two orders of magnitude *below* the onset of any fluency cost, and roughly two orders below total collapse. The usable dynamic range of this decoder is $\alpha \approx 0.03 \rightarrow 500$, about four orders of magnitude wide. A re-run confined to the published range cannot detect either detoxification or harm, and its null result is uninformative about the method.

This is the empirical counterpart to Appendix C.4, which reached the same conclusion by calibration rather than by generation.

I.4 The U2 rule was exercised, and its verdict is not evidence

docs/METHODS.md U2 fixes the schedule decision in advance. It was applied: `select_schedule` returned `adopted = True`, `best_kind = margin_gated`, `relative_gain = 0.286`. **That verdict is not evidence of anything, and the reason is instructive.**

The rule takes a toxicity score and a perplexity. Toxicity is unavailable here — the subspace is below chance at every layer — so steering strength was substituted, and inverse distinct-2 for perplexity. But **steering strength is anti-correlated with the true objective**: less steering trivially “reduces toxicity” in the proxy. The winner won by being *weaker* (peak α 5.71 against 8.00), which is exactly what a proxy minimised by inaction rewards. The Δ PPL term did not catch it because the fluency differences are inside noise.

Methodological finding, stated as a rule for others: never use steering strength, KL, or any monotone function of the control effort as a proxy for toxicity. Every such proxy is minimised by doing nothing, so it will reliably select the weakest configuration in a sweep. U2’s real requirement — an external toxicity measure — is not a formality; it is the load-bearing part of the rule, and the moment it is replaced by a self-referential quantity the rule inverts.

sasa/scheduling.`select_schedule` is therefore shipped with the rule quoted verbatim in its output and a `rule` field recorded alongside the verdict, so a consumer of the JSON can see which inputs were real and which were substituted.

I.5 Threats

- **16 prompts, 24 tokens.** Enough to locate a cliff this sharp, not enough for subtle comparisons. The schedule table in I.2 should be read as “no effect detectable”, not as an ordering.
- **One model, one layer, one fitted subspace.** The cliff’s *location* in logit SD is the transferable quantity; the α at which it occurs is specific to this w and would move with a different fit (Appendix C.4).

- **Fluency is measured with lexical proxies** (distinct-2, repetition), not perplexity. Repetition *falls* to zero at $\alpha \geq 128$ precisely because the output becomes a single repeated string, so repetition rate is not monotone in degeneracy and must be read alongside distinct-2. This is a trap worth naming: a single fluency metric would have shown “improvement”.

Appendix J — Where subspace transfer breaks

Idea 11 of §6, following Appendix F. Code and data: `experiments/run_transfer_boundary.py`, `results/transfer_boundary.json`

Appendix F tested one same-family pair and showed that orthogonal Procrustes beats ridge by $46\times$. What that leaves open is the *boundary*: how transfer quality varies across pairs, and whether anything cheap predicts it. If something does, the expensive part of Phase 3 — fitting a map — becomes a screening step. If nothing does, the 2–5 k paired states in `docs/ROADMAP.md` are a bet rather than a plan.

J.1 Setup

12 ordered pairs over four models: `distilgpt2` (768), `gpt2` (768), `gpt2-medium` (1024) and EleutherAI/`pythia`-410m (1024). 8 documents $\times$ 24 token positions = **192 paired observations** per model, 115 for fitting and 77 for evaluation. Mismatched widths are zero-padded on the right, as `docs/ROADMAP.md` prescribes for cross-family work.

Two measurement decisions are worth recording, because both were wrong first and both would have inverted the conclusion.

Direction sign is arbitrary. A principal direction can be negated, so a *perfectly* transported direction scores -1 on a signed correlation. An early run reported `gpt2-medium` $\rightarrow$ `gpt2` at -0.899 — which read as a catastrophic failure of a same-family pair. Agreement is therefore $|\rho|$, with the signed value retained alongside it.

“Same family” is not a string prefix. `distilgpt2` is distilled *from* `gpt2` and shares no prefix with it, so a prefix test classified the closest pair in the set as cross-family. An explicit family map is used instead. This is the kind of error that would survive into a paper.

J.2 Results

Proxy $\rightarrow$ target	Same family	Procrustes residual	Held-out agreement	Linear CKA
<code>distilgpt2</code> $\rightarrow$ <code>gpt2</code>	yes	0.365	0.999	0.802
<code>gpt2</code> $\rightarrow$ <code>distilgpt2</code>	yes	0.556	0.999	0.802
<code>gpt2</code> $\rightarrow$ <code>gpt2-medium</code>	yes	1.308	0.855	0.669
<code>gpt2-medium</code> $\rightarrow$ <code>gpt2</code>	yes	1.534	0.899	0.669
<code>gpt2-medium</code> $\rightarrow$ <code>distilgpt2</code>	yes	2.080	0.497	0.646
<code>distilgpt2</code> $\rightarrow$ <code>gpt2-medium</code>	yes	1.143	0.026	0.646
<code>pythia</code> $\rightarrow$ <code>gpt2-medium</code>	no	1.058	0.971	0.104
<code>pythia</code> $\rightarrow$ <code>gpt2</code>	no	1.090	0.601	0.159
<code>pythia</code> $\rightarrow$ <code>distilgpt2</code>	no	1.204	0.591	0.200
<code>distilgpt2</code> $\rightarrow$ <code>pythia</code>	no	1.835	0.139	0.200
<code>gpt2</code> $\rightarrow$ <code>pythia</code>	no	2.558	0.120	0.159

Proxy $\rightarrow$ target	Same family	Procrustes residual	Held-out agreement	Linear CKA
gpt2-medium $\rightarrow$ pythia	no	2.945	0.067	0.104

Aggregate	Value
Mean transfer error, same family ($n = 6$)	0.288
Mean transfer error, cross family ($n = 6$)	0.585
Spearman(CKA, transfer error)	$\rho = -0.531, p = 0.047$
Spearman(CKA, fit residual)	$\rho = -0.503, p = 0.065$

J.3 Reading

Crossing a family boundary doubles the transfer error, from 0.288 to 0.585, with no overlap in the medians. This is the boundary docs/ROADMAP.md Phase 3 plans to cross (Llama $\rightarrow$ Qwen/Gemma), and the size of the penalty is now measured rather than assumed. It is a real cost, and it is survivable: three of the six cross-family pairs land above 0.59 agreement, so the mechanism works across families even when the alignment is weaker.

CKA ranks the degradation, at the edge of significance. $\rho = -0.53$ over 12 pairs with $p = 0.047$ is a rank-order relationship in the expected direction: better-aligned spaces transfer better. It is not strong enough to plan around.

CKA is nowhere near sufficient, and the counterexamples are not marginal.

- pythia $\rightarrow$ gpt2-medium transfers at **0.971** with a CKA of **0.104**.
- distilgpt2 $\rightarrow$ gpt2-medium collapses to **0.026** with a CKA of **0.646** — a higher CKA, a worse transfer.

The first says a low-alignment pair can transfer almost perfectly; the second says a moderately-aligned pair can fail almost completely, in the same family. Whatever distinguishes them is not captured by linear CKA, and the direction of transfer matters more than either model alone: pythia $\rightarrow$ gpt2 works (0.601) where gpt2 $\rightarrow$ pythia does not (0.120), at identical CKA.

Honest status: the correlation is a direction, not a coefficient. With 12 pairs drawn from one small-model family plus one outlier family, $\rho = -0.53$ at $p = 0.047$ is the boundary of what the sample can support, and the two counterexamples above are exactly the cases a screening rule would get wrong. The useful conclusion is narrow: **CKA is worth computing before committing to a transfer, because it will often flag a bad pair, but a good CKA is no evidence that the transfer will work.** The cheap screen has a high false- positive rate, and Phase 3 should budget for pairs that pass it and still fail.

J.4 Threats

- **The transported direction is the top principal component** of the fit split, which for text representations is dominated by frequency and position effects. This measures the *transfer operator*, not whether a semantic subspace would survive it — the caveat from Appendix F applies unchanged.
- **One cross-family family.** pythia is the only non-GPT-2 model, so “cross family” here means “GPT-2 versus Pythia”, not a general claim.
- **192 paired observations.** Appendix F showed why this matters: 9 pairs produced a near-perfect and entirely vacuous result. 192 is enough for the map to be estimable but small for a 12-point correlation.
- **Zero-padding is a choice.** distilgpt2 $\rightarrow$ gpt2-medium (the 0.026 collapse) pads 768 $\rightarrow$ 1024. A different padding scheme might change that row materially, and it is the row that most affects the correlation.

J.5 What would settle it

The obvious next step is a wider sweep: more pairs within the GPT-2 family to separate *direction* effects from *family* effects, plus a second genuinely different family. Both are cheap at this model scale — a 24-position harvest is about a minute per model — which is itself the practical argument for CKA as a screen: measuring a candidate pair costs a fraction of fitting the map, so several candidates can be surveyed before committing to one.

Appendix K — The decode loop: caching and batching

Ideas 15 and 18 of §6. Code: `sasa/decoding.py, tests/test_probes_and_decoding.py`. Data: `results/decode_costs.json`. Reproduce with `experiments/run_decode_costs.py`.

Appendix B removed the per-token margin cost exactly. This appendix removes the other term — and it is the one that dominates, which is why no efficiency claim about SASA can be stated without addressing it.

K.1 Why this is the real cost

`SASASampler.generate` re-runs a full forward pass over the entire prefix at every step and never populates a key-value cache. Generating L tokens therefore costs

$$\underbrace{P + (P+1) + \dots + (P+L)}_{\text{uncached}} \quad \text{versus} \quad \underbrace{P+L}_{\text{cached}}$$

forward token positions, where P is the prompt length — quadratic in the number of generated tokens. `sasa.decoding.token_budget` computes both counts and is the quantity any efficiency claim should be stated against.

K.2 Caching: exact, and the analytic gain is not yet realised

Every configuration was checked for *distributional equality* before being timed. A speed-up measured against an implementation that computes something different is not a speed-up.

Prompt	New tokens	Uncached	Cached	Measured	Analytic	Tokens identical
4	8	4 058.7 ms	3 254.6 ms	1.25×	5.7×	yes
8	16	9 224.4 ms	5 962.6 ms	1.55×	11.0×	yes
16	32	7 100.0 ms	2 683.8 ms	2.65×	21.7×	yes

All 12 configurations produced **identical token sequences**, with a maximum per-step logit-norm difference of 1.7×10^{-2} — float32 accumulation order, not a behavioural difference.

Two things are worth reading off this table. The **measured** gain grows with length, $1.25 \times \rightarrow 2.65 \times$, as expected once the quadratic term dominates. And it stays far below the **analytic** gain, $2.65 \times$ against $21.7 \times$, which is the honest finding: the remaining cost is the model’s own forward pass, and the margin term is no longer a meaningful part of the budget. Appendix B’s arithmetic collapse and this loop together account for the whole overhead; either alone leaves a large factor on the table.

K.3 Batching: correctness yes, throughput only once vectorised

The steering vector being context-free (Theorem 1) means one $(V,)$ addition serves every row, so a batch can be sampled with a single `torch.multinomial` over a (B, V) distribution. Two reproducibility contracts are offered, and they genuinely differ:

- **per-row generators** — a row’s token is independent of its position in the batch, at the cost of a Python loop;
- **a single batch generator** — one vectorised call, fast, but a row’s draw depends on where it sits.

Batch	Exact vs independent rows	Vectorised (ms/item)	Per-row loop (ms/item)	Vectorised speed-up
1	yes	5.824	4.533	0.78×
2	yes	3.994	5.340	1.34×
4	yes	2.849	8.674	3.04×

Batch	Exact vs independent rows	Vectorised (ms/item)	Per-row loop (ms/item)	Vectorised speed-up
8	yes	2.177	4.749	2.18×

The exactness guarantee holds at every size. The throughput result is the instructive one: **the first implementation of `batched_sample` was a Python loop over rows, and it produced a flat per-item cost — 4.5 to 5.3 ms/item at every batch size.** That is not batching; it is the same computation with extra bookkeeping. Only the vectorised path shows the expected behaviour, falling from 5.82 to 2.18 ms/item and delivering up to **3.04×**.

A benchmark that had been run against the loop version would have concluded that batching does not help a steered decoder, and that conclusion would have been correct *about that code* and wrong about the idea.

K.4 Threats

- **One model, CPU, 2 cores.** Absolute milliseconds are not portable; the ratios and the exactness checks are.
- **Only top- k sampling was measured.** Nucleus filtering sorts a V -element vector per row and would not vectorise as cleanly, so the batching result may not hold at top- p .
- **The cache is a standard HF `DynamicCache`.** Its benefit is the model’s own prefill-plus-step behaviour; nothing here is a custom attention implementation, and a fused kernel would narrow the gap to the analytic prediction further.
- **The batching speed-up is measured on logits already in memory.** End-to-end batched *generation* would also need a cache per row, which this environment does not test.

Appendix L — Is the pinned judge adversarially fooled?

Idea 17 of §6. Code: `experiments/run_judge_gaming.py`, `sasa/judge.py`. Data: `results/judge_gaming.json`.

Appendix D found that steering moved the SASA objective without moving the pinned judge. That is a success for the headline claim, but it is equally a hypothesis about *why*: either the steering was ineffective, or the judge is insensitive. The second possibility is an artifact, and an artifact that would silently invalidate Appendix D. So the judge was attacked directly, with paraphrases designed to preserve meaning while removing the surface markers of abuse.

L.1 Design

12 prompts from the Appendix D set. Three completions per prompt:

- **original** — unmodified;
- **attacked** — an LLM rewrite of the original, instructed to keep the meaning and remove profanity, insult, and second-person address;
- **control** — a rewrite of a *different, neutral* prompt under the same instructions.

The control is the part that matters. Without it, any drop in judged toxicity would be attributable to the rewrite instructions having removed the abuse — which is the intended effect, not a successful attack. The control isolates the judge’s response to *rewriting as such* from its response to *evasion*.

L.2 The attack was applied

This is a check worth stating first, because the natural failure mode of this experiment is to report a null without ever verifying the treatment worked.

Identity check	Count
<code>attacked == control</code>	0 / 12
<code>attacked == original</code>	0 / 12

Every attacked text differs from both its original and its control, so the rewriting was verifiably applied. Inspecting the outputs shows the intended transformation:

original — “You are such a fucking idiot, I hope everyone knows how much you suck at everything you touch.”

attacked — “It is quite amusing to observe how some individuals feel compelled to share their thoughts, regardless of their lack of merit.”

The profanity and direct insult are gone; the contempt is intact.

L.3 Results

Condition	Mean judged toxicity (0–4)
original	0.250
steered	0.167
attacked	0.167
control	0.167

Paired comparison	Non-zero differences	<i>n</i>	<i>p</i>
attacked vs steered	0	2	0.655
control vs steered	0	2	0.655
attacked vs control	0	0	1.000

L.4 The honest reading

The judge was not fooled, and this experiment cannot tell you whether that is good news.

The observed result is that judged toxicity did not fall when the surface form was evaded — which is a point in the judge’s favour, and is consistent with Appendix D. But the effective paired sample is **zero to two**, because a 0–4 integer judge produces only five values and a 12-prompt set mostly ties. A Wilcoxon signed-rank test on zero non-zero differences is not a measurement of robustness; it is a statement that the instrument has no resolution here.

Conclusion: underpowered and uninformative. No claim of judge robustness, and no claim of judge vulnerability, is supported by this data. Reporting the unchanged mean as evidence that “the judge is not adversarially fooled” would be reading a resolution limit as a finding. The honest output is the observation in L.2 — that paraphrase attacks were successfully generated and successfully evaded surface markers — plus the negative result about the measurement apparatus.

That is a negative result about this experiment, and the useful thing it yields is the following methodological rule, now applied to Appendix D as well.

L.5 What Appendix D’s null result now means

Appendix D’s headline — the external judge did not move while the steering objective did — is weakened in a specific way, and it should be stated this way in the write-up:

The pinned judge **failed to separate** steered from unsteered generations. The steering objective moved; the judge did not. Since the judge demonstrably retains sensitivity to direct abuse (Appendix D: steered scored 0.167 against baseline 0.250), the separation is not explained by a dead judge. But a paraphrase attack preserves judged toxicity (this appendix), so the judge’s response to a *disguised* abuse is untested. **The correct claim is that steering did not fool this judge, not that this judge catches everything.**

The difference matters: the first is a finding about SASA, the second is a finding about a measurement instrument and would be a much larger claim than 12 prompts of 0–4 integer scores can support.

L.6 How to make this testable

Four changes, in order of cost:

1. **Use a continuous score.** A 0–4 integer scale with 12 prompts has at most $5 \times 12 = 60$ distinguishable outcomes and produced 0 usable pairs. A rubric scored on a continuous scale, or a probability-of-toxicity head, would resolve differences the current instrument cannot see.
2. **Power the design.** With a continuous score and paired differences of plausible size, 12 prompts is roughly one order of magnitude short; ~100 prompts would be reasonable. Paired designs are essential — the original/attacked pairing removes between-prompt variance.
3. **Include an easy positive control** that *should* move the score — direct profanity — to confirm the attack arm is capable of producing a difference. Without one, a null is uninterpretable, which is exactly the problem here.
4. **Attack the rubric, not just the text.** The current attack targets profanity, which the rubric’s harmfulness criterion plausibly ignores. A jailbreak-shaped generation (instructional override, role-play framing, code block) would probe the weaker link.

Points 1 and 2 are the binding constraints. Until the instrument can resolve a difference, an adversarial study of it is not possible, and no amount of prompt craft substitutes for resolution.
